

UNIT - VI

6

Ruby and Rails

Syllabus

Introduction to Ruby : *Origins & uses of Ruby, scalar types and their operations, simple input and output, control statements, fundamentals of arrays, hashes, methods, classes, code blocks and iterators, pattern matching.* **Introduction to Rails :** *Overview of Rails, Document Requests, Processing Forms, Rails Applications and Databases, Layouts, Rails with Ajax. Introduction to EJB.*

Contents

- 6.1 Origins & Uses of Ruby
- 6.2 Scalar Types and their Operations
- 6.3 Simple Input and Output
- 6.4 Control Statements
- 6.5 Fundamentals of Arrays
- 6.6 Hashes
- 6.7 Methods
- 6.8 Classes
- 6.9 Code Blocks and Iterators
- 6.10 Pattern Matching
- 6.11 Overview of Rails
- 6.12 Document Requests
- 6.13 Processing Forms
- 6.14 Rails Applications and Databases
- 6.15 Layouts
- 6.16 Rails with Ajax
- 6.17 Introduction to EJB **May-18, 19, Dec.-19, Marks 8**
- 6.18 Multiple Choice Questions

Part I : Introduction to Ruby

6.1 Origins & Uses of Ruby

- Ruby is a scripting language designed by Yukihiro Matsumoto also known as Matz.
- Ruby executes on variety of platforms such as Windows, Linux, Mac and so on.
- Ruby is a pure object oriented programming language. Ruby is an open source and it is freely available on the web.
- Ruby is similar to smalltalk, python, Perl.
- Ruby has the pattern matching feature which is similar to the Perl.
- Ruby can be easily connected to the databases like MYSQL, DB2 and Oracle.

Environment

- For executing the Ruby programs we need to install **Rubystack**.
- The Ruby Stack can be downloaded and installed from <http://bitnami.com/stack/ruby>.
- It can be downloaded on Windows/ Linux or MacOS platform. It consists of Ruby, Rails, Apache, MySQL, SQLite, Git, PHP and phpMyAdmin in one pack. Installing Ruby Stack is quick and easy. It is a **self - contained** system.
- After installing the Ruby Stack we can start the Apache Server and MYSQL Server using the **Managers**.

6.2 Scalar Types and their Operations

- There are three types of data types used in Ruby and those are Scalar, Arrays and hashes.
- The scalar types are further classified as **numeric** and **character strings**. Refer Fig. 6.2.1.

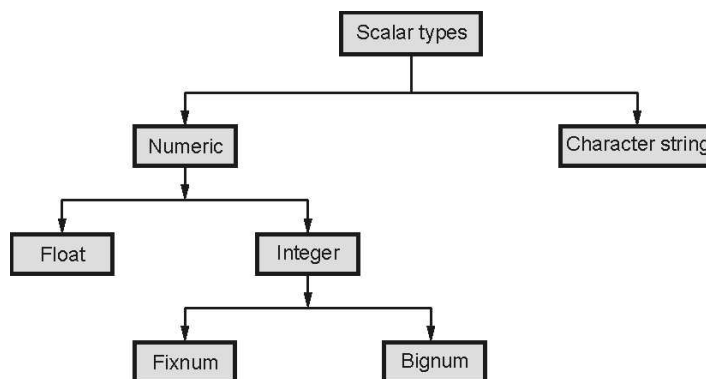


Fig. 6.2.1 Scalar types

6.2.1 Numeric and String Literals

- All the numeric data types in Ruby are derived from the **Numeric** class.
- The immediate children of Numeric class are **Float** and **Integer**.
- The **Integer** class has two child classes - **Fixnum** and **Bignum**.
- The integer objects which lies within the size of 32 bit is called the **Fixnum** object and the object which is greater in size of 32 bit is called the **Bignum** object.
- If a **Fixnum** object whose size grows beyond the 32 bit then it is implicitly typecast to **Bignum** object.
- Similarly in expression evaluation the size of particular Bignum object becomes less than 32 bit then it is converted to the **Fixnum** object.
- The integer literal may contain the underscore character to read out the large integer conveniently. For example 231456981 can be read using the underscore as 231_456_981.
- The numeric literal that contains the floating point is called the **Float** object. It denotes the double precision floating type value.
- The decimal point must be preceded and followed by at least one digit. For example .22 will be illegal literal in Ruby.
- The **string** object consists of the string literals that are nothing but the collection of characters.
- There are two types of strings - single quoted and double quoted string.
- If we want to print some character literally like apostrophe s then the \ is used.

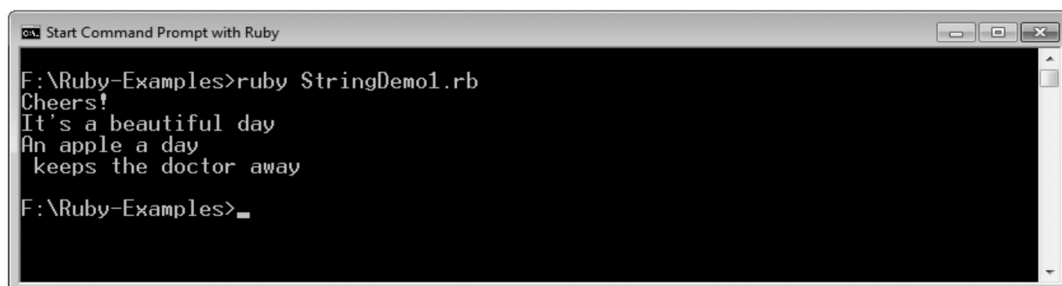
Following is a simple script written in Ruby to demonstrate the use of string literal

Step 1 : Open the Notepad or some suitable text editor and type the following code -

Ruby Document [StringDemo1.rb]

```
puts 'Cheers!'
puts 'It\'s a beautiful day'
puts "An apple a day \n keeps the doctor away"
```

Step 2 : Open the command editor for ruby and you can see the following output -



```
Start Command Prompt with Ruby
F:\Ruby-Examples>ruby StringDemo1.rb
Cheers!
It's a beautiful day
An apple a day
keeps the doctor away
F:\Ruby-Examples>
```

6.2.2 Variable and Assignment Statements

- The variable used for storing the data values.
- The variables in Ruby are **case sensitive**.
- It may contain the letters, underscore and digits. Conventionally the variable must be declared in the lower case.

Ruby Script [VariableDemo.rb]

```
pi =3.14
r=15
puts "The value of pi variable is #{pi}"
puts "The radius of the circle is #{r}";
puts "The area of the circle is #{pi*r*r}"
```

Output

```
The value of pi variable is 3.14
The radius of the circle is 15
The area of the circle is 706.5
```

- There is no need to declare the variable in Ruby.
- In Ruby every data value is an object. Hence objects need references.
- If a scalar variable is not assigned with a value then it is assigned with the value **nil**.
- The constants in Ruby are declared with the first letter capital. A constant is created when a variable is assigned with some value it becomes a constant variable.
- There are special variables in variables which are called as **implicit variables**. The scalar variables that are implicit variables begin with dollar sign while other type of implicit variables can begin either by underscore `_`, caret `^` or back slash `\`
- The variables in Ruby are **typeless**. All that is assigned in an assignment statement is address of an object.

6.2.3 Numeric Operators

- The arithmetic operators such as `+` addition, `-` subtraction, `*` multiplication `/` division and `mod` operators.
- The **precedence rules** specify which operator is to be evaluated first when to operators of different level of precedence appear in the expression.
- The **associativity rule** specify which operator is to be evaluated first when two operators of same level of precedence appear in the expression.
- The Ruby does not include increment `++` or decrement `--` operator.

- There is a **math** module supported by Ruby. This module provides the methods for basic trigonometric and mathematical functions. For instance the functions such as cos, sin, tan, sqrt are supported by this module.
- Along with the software package of Ruby an interactive interpreter is provided. This interpreter is called **Interactive Ruby**. The acronym for this interpreter is **irb**. The user can type the expression and get the answer of this expression immediately.
- For that purpose, click on Start->All Programs->Ruby 1.9.2->Interactive Ruby. Following is the illustration -



```
Interactive Ruby
irb(main):001:0> 5+7
=> 12
irb(main):002:0> Math.sqrt(25)
=> 5.0
irb(main):003:0> 21/7
=> 3
irb(main):004:0>
```

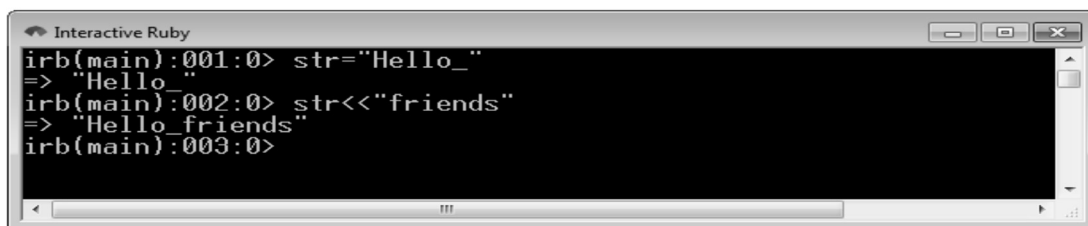
6.2.4 String Methods

- Ruby supports various string methods.
- String method supports concatenation operator. The operator is denoted by +. The operator + is used for concatenation.



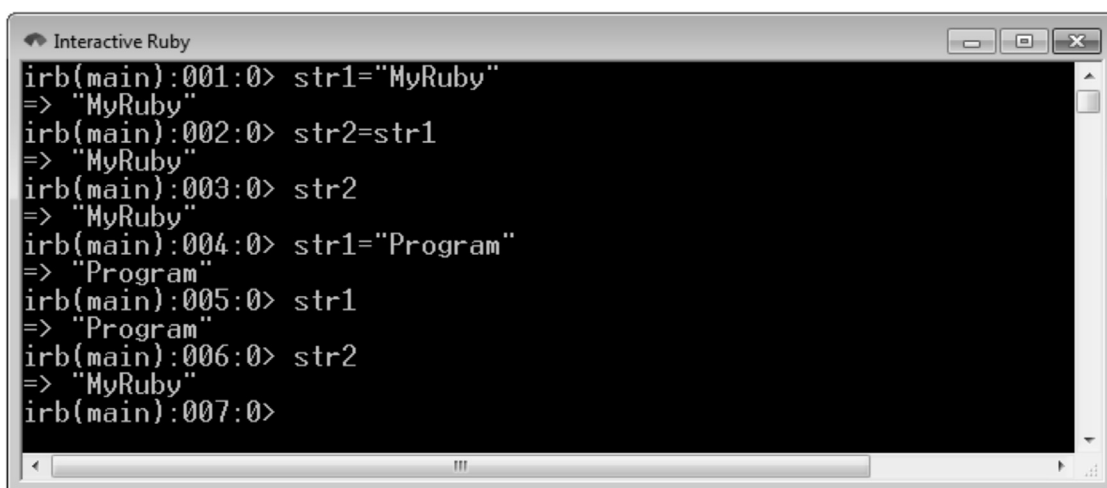
```
Mark Interactive Ruby
irb(main):001:0> "Hello"+"###"+"India"
=> "Hello###India"
irb(main):002:0>
```

- The operator << is used to append the string. For example :



```
Interactive Ruby
irb(main):001:0> str="Hello_"
=> "Hello_"
irb(main):002:0> str<<"friends"
=> "Hello_friends"
irb(main):003:0>
```

- Every variable in Ruby refers to some memory location. If one variable is assigned to another variable then that variable will refer to the same memory location. For example :



```
Interactive Ruby
irb(main):001:0> str1="MyRuby"
=> "MyRuby"
irb(main):002:0> str2=str1
=> "MyRuby"
irb(main):003:0> str2
=> "MyRuby"
irb(main):004:0> str1="Program"
=> "Program"
irb(main):005:0> str1
=> "Program"
irb(main):006:0> str2
=> "MyRuby"
irb(main):007:0>
```

For example : In above screen-shot you can see that the variable str1 contains the string "MyRuby". Another variable str2 is assigned with the value of str1. Hence on typing the string str2 we get the same string i.e."MyRuby". Now the string str1 is assigned with another string "Program". Hence we get str1="Program" but still the value of str2="MyRuby".

- Instead of directly assigning some string to str1, we can use the **replace** method for assigning some new string.

For example: Instead of
str1="Program"
we can write
str1.replace("Program");

- Following are some methods that are used for various string operations -

Method/Operator	Action	Example
Capitalize	This method converts the first letter to uppercase	"india".capitalize => "India" But "11a".capitalize => "11a"
chop	This method removes the last character.	"Lucky".chop => "Luck"
chomp	This method the newline from the right end, if there is one.	"Hello\n Friends\n".chomp => "Hello\n Friends"
equal ==	The method equal is used to compare two objects. The == equal to operator treats the string literally.	"Nisha".equal?("Nisha") => false because these are treated as two different objects "Nisha"=="Nisha" => true because the string is compared literally
upcase	This method converts all the lower case letters to the object case.	"nisha".upcase => "NISHA"
downcase	This method converts the upper case letters of the string to lower case.	" NISHA ".upcase => "nisha"
strip	This method removes the space on both ends.	" India ".strip => "India"
lstrip	This method removes the space from the left end	" India ".lstrip => "India "
rstrip	This method removes the space from the right end	" India ".strip => " India"
swapcase	This method converts the upper case to lower case and lower case to upper case.	"NiSha".swapcase => "nIsHA"
reverse	It reverses the string.	"Ruby".reverse => "ybuR"
*	The repetition operator is used to specify the repetition. The repetition is specified by asterisk(*)	"Hurray!!! "*3 => "Hurray!!! Hurray!!! Hurray!!! "
<=>	This is called the spaceship operator. If two operands are equal then it returns the 0; If first operand is greater than the second operand then it returns 1; If the second operand is greater than the first one then it returns -1	"aaa"<=>"bbb" => -1 "bbb"<=>"aaa" => 1 "aaa"<=>"aaa" => 0

6.3 Simple Input and Output

- The input and output operations are the most important operations for any programming language.
- Typically the input is given by the keyboard and the output is displayed on the screen.

6.3.1 Screen Output

- The **puts** method is used to display the output on the screen.
- The operand for the **puts** method is **string**.
- The value is the variable can be displayed using the **puts** method but in that case `#{...}` is used. For example :

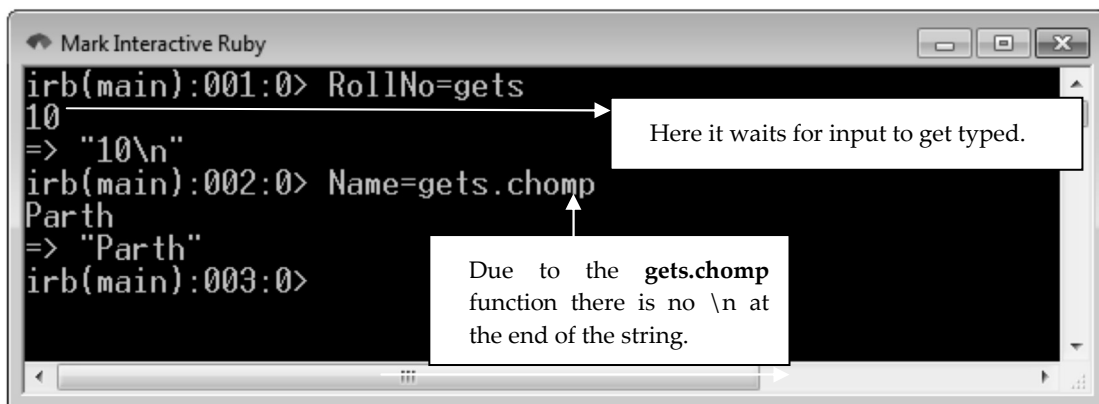
```
RollNo=10
Puts "My Roll number is #{RollNo}"
```

- The new line character can be appended at the end of the string so that next subsequent contents can be displayed on the screen. But it writes the word **nil** at the end. For example :

```
puts "My favorite langauge is #{str}"
My favorite langauge is Ruby
=> nil
```

6.3.2 Keyboard Input

- In Ruby the keyboard input is rarely needed but we can give the input to the program using keyboard.
- For the keyboard input the method **gets** is used. This method reads the line of input.



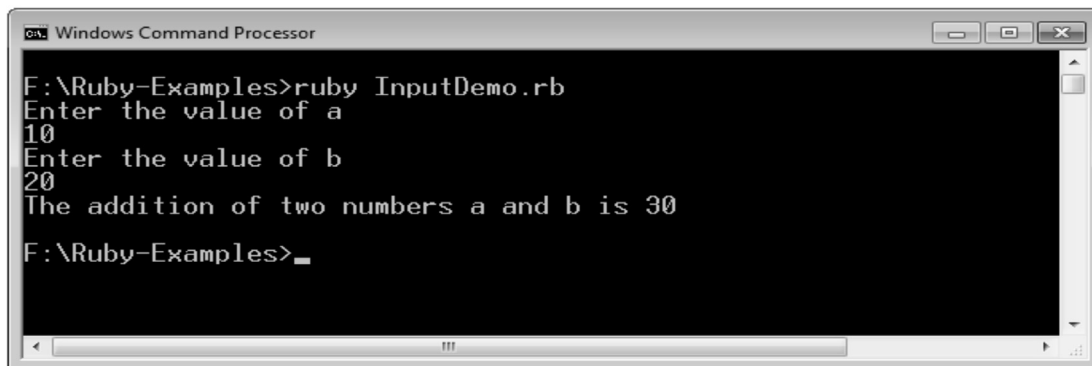
- The **gets.to_i** or the **gets.to_f** method is useful for obtaining the integer or the float value. Following are the simple steps that can be followed for executing the ruby program :

Step 1 : Open the suitable text-editor such as Notepad or Wordpad. Type the following code and save this file using the extension **.rb**.

InputDemo.rb

```
puts "Enter the value of a"
a=gets.to_i
puts "Enter the value of b"
b=gets.to_i
c=a+b
puts "The addition of two numbers a and b is #{c}"
```

Step 2 : Now type the command at the command prompt for executing the ruby program as **Ruby InputDemo.rb**



```
Windows Command Processor
F:\Ruby-Examples>ruby InputDemo.rb
Enter the value of a
10
Enter the value of b
20
The addition of two numbers a and b is 30
F:\Ruby-Examples>_
```

6.4 Control Statements

- In Ruby the control statements are used for controlling the flow of control.
- It consists of control expressions, selection statements and looping statements.

6.4.1 Control Expressions

- The control expressions are Boolean type expressions.
- The relational expression has two operands and one operator.
- There are two sets of Boolean operators. The operators **&&(AND)**, **|| (OR)**, and **!(NOT)** are of highest precedence while the operators **and**, **or** and **not** are of lower precedence.

- The operators are enlisted in the following table -

Operator	Meaning
==	Equal to operator. If two operands are equal then it returns true.
!=	Not equal to. If the two operands are not equal then it returns true.
<	The less than
>	The greater than
<=	Less than equal to
>=	Greater than equal to
<=>	Compares the two operands
eql?	It returns true if the receiver object and the parameter both have the same type and values.
Equal?	Compares the two objects.

- The associativity of the operators is as shown in the following table. The highest precedence operators are listed first and then the lowest precedence operators.

Operator	Description	Associativity
**	Exponentiation	Right
! ~ unary + unary -	Not, complement, unary plus and minus	Right
* / %	Multiply, divide and modulo	Left
+ -	Plus and minus	Left
&	Bitwise 'and'	Left
<= < > >=	Comparison operators	Not associative
<=> == === != =~ !~	Equality and pattern match operators (!= and !~ may not be defined as methods)	Not associative
&&	Logical 'and'	Left
	Logical 'or'	Left

<code>=, %=, /=, +=, -=, &=, >>=, <<=, *=, &&=, =, **=, -=</code>	Assignment	Right
Not	Logical negation	Right
or and	Logical composition	Right

6.4.2 Selection and Looping Statements

Explain how multiple selection constructs are implemented in Ruby.

- The control statements are for controlling the flow of control of the program.
- The **if statement** is of following form -

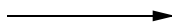
Syntax:
if expression
Statement
end



Example:
if *count* < 10
 puts "Hello"
end

- The **if else statement** is -

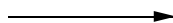
Syntax:
if expression
 Statement
elsif *expression*
 Statement
else
 Statement
end



Example:
if *age* < 5
 puts "Kid"
elsif *age* < 21
 puts "Youth"
else
 puts "Adult"
end

- Ruby allows two kinds of **multiple selection constructs**. Both named **case**. The first form of **case** statement is as follows -

Syntax:
case
when *value* **then**
 statement sequence
when *value* **then**
 statement sequence
 ...
else *Expression*
end



Example:
case
when 66 **then**
 puts "Distinction"
when 60 **then**
 puts "First Class"
when 50 **then**
 puts "Pass Class"
else *puts "Fail"*
end

- Another form of **case** statement

```
case
when Boolean expression then
  statement sequence
when Boolean expression then
  statement sequence
...
else Expression
end
```

- The **while** and **for** statements are similar to C language. These are as follows -

Syntax:
while *control expression*
 Loop body
end

Example:
while *count*<10
 puts "The count =#{count}"
 count=*count*+1
end

- The **for** statement is as follows -

Syntax:
for *variable* [, *variable* ...] **in** *expression* [**do**]
 Loop body
end

Example:
for *i* **in** 0..10
 puts "Value of *i* is #{i}"
end

- The **until** statement is also used for looping. The body of **until** statement executes when the condition is false.

Syntax

```
until conditional [do]  

  loop body statements  

end
```

Ruby Script[UntilDemo.rb]

```
$i = 0;  

$n = 3;  

until $i > $n do  

  puts("Inside loop with i = #$i" );  

  $i +=1;  

end  

puts("Outside the loop with i=#$i")
```

Output

```
Inside loop with i = 0  

Inside loop with i = 1  

Inside loop with i = 2  

Inside loop with i = 3  

Outside the loop with i=4
```

The **break** statement is used to terminate the current execution. For example :

Ruby Script[BreakDemo.rb]

```
for count in 0..7
  if count > 2*2 then
    break
  end
  puts "count= #{count}"
end
```

Output

```
count= 0
count= 1
count= 2
count= 3
count= 4
```

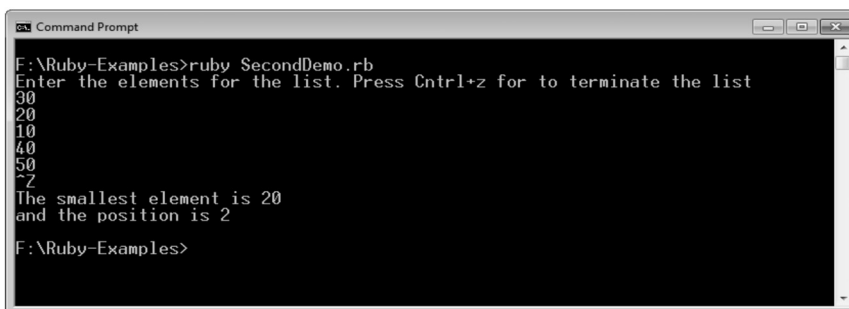
Example 6.4.1 *Write a Ruby program that inputs the list of numbers from keyboard and finds the second smallest number in the list, along with its position in the list.*

Solution :

Ruby Program[SecondDemo.rb]

```
i=0
arr=Array.new
puts "Enter the elements for the list. Press Cntrl+z for to terminate the list"
while(val=gets)
  arr[i]=val
  i+=1
end
first = arr[0]
second = arr[0]
i=0
for value in arr
  if value < first
    second = first
    secondkey=i
    first = value
  elsif value < second
    second = value
    secondkey=i
  end
  i+=1
end
puts "The smallest element is #{second} and the position is #{secondkey}"
```

Output



```
Command Prompt
F:\Ruby-Examples>ruby SecondDemo.rb
Enter the elements for the list. Press Cntrl+z for to terminate the list
30
20
10
40
50
7
The smallest element is 20
and the position is 2
F:\Ruby-Examples>
```

6.5 Fundamentals of Arrays

- In Ruby the arrays are more flexible because of two reasons -
 - The arrays in Ruby are **dynamic** that means the array can grow and shrink.
 - The arrays in Ruby can store the elements of different data types.
- The arrays can be created in two different ways.
- The first way is to use the keyword **new**. For example :
 - Marks =Array.new
 - Marks=Array.new(10) ← the array of 10 locations gets created.
- The second way is to assign the list of literals to the variables. For example :
 - Student=[10,20,30,"Archana",[]]
- The subscript of array elements is integer and every array starts at the subscript 0.
- The **length** method is used to retrieve the length of the array.
- The **at** method returns the element present at the pointed subscript.

Ruby Script [ArrayDemo.rb]

```
A=[10,20,30,40,50]
puts "The array is : #{A}"
num=A.at(3)
puts "The element at 3rd index is :#{num}"
len=A.length
puts "The length of the array is :#{len}"
```

Output

```
The array is : [10, 20, 30, 40, 50]
The element at 3rd index is :40
The length of the array is :5
```

6.5.1 The for-in Statement

- The for-in statement is used to visit the elements of an array. Following is a simple example that illustrates the use of for-in statement -

Ruby Script [ForInDemo.rb]

```
A=[10,20,30,40,50]
for i in [0,1,2,3,4]
  puts "For index: #{i}, value=#{A[i]}"
end
```

Output

```
For index: 0, value=10
For index: 1, value=20
For index: 2, value=30
For index: 3, value=40
For index: 4, value=50
```

6.5.2 Built in Methods for Arrays and Lists

- In this section, we use various built in methods for supporting array operations. Open the Interactive Ruby interpreter and invoke various methods.
- The **shift** method removes the first element of an array. For example :

```
irb(main):001:0> A=[10,20,30,40,50]
=> [10, 20, 30, 40, 50]
irb(main):002:0> num=A.shift
=> 10
irb(main):003:0> A
=> [20, 30, 40, 50]
irb(main):004:0>
```

- The **push** method is used to push the elements from the end in the array. For example :

```
irb(main):001:0> A=[10,20,30]
=> [10, 20, 30]
irb(main):002:0> A.push(40,50)
=> [10, 20, 30, 40, 50]
irb(main):003:0>
```

- The two arrays can be combined together using the **+** operator. For example :

```
irb(main):001:0> A=[10,20,30]
=> [10, 20, 30]
irb(main):002:0> B=[1.1,1.2,1.3]
=> [1.1, 1.2, 1.3]
irb(main):003:0> C=A+B
=> [10, 20, 30, 1.1, 1.2, 1.3]
irb(main):004:0>
```

- The **reverse** method is used to reverse the contents of the array. For example :

```
irb(main):001:0> A=[10,20,30,40,50]
=> [10, 20, 30, 40, 50]
irb(main):002:0> A.reverse
=> [50, 40, 30, 20, 10]
irb(main):003:0> A
=> [10, 20, 30, 40, 50]
irb(main):004:0>
```

- The **sort** method is used to sort the contents of the arrays. For example

```
irb(main):001:0> A=[30,10,20,50,40]
=> [30, 10, 20, 50, 40]
irb(main):002:0> A.sort
=> [10, 20, 30, 40, 50]
irb(main):003:0>
```

- The **include?** is used to check whether the array contains particular element or not.

```
irb(main):001:0> A=[10,20,30,40]
=> [10, 20, 30, 40]
irb(main):002:0> A.include?(20)
=> true
irb(main):003:0> A.include?(99)
=> false
irb(main):004:0>
```

6.5.3 Examples

Following is a simple Ruby program that sorts the array and then searches the desired element from the array.

Ruby Program

```
i=0
A=Array.new
puts "Enter the elements in the array"
while(i<5)
  val=gets
  A[i]=val
  i+=1
end
A.sort!
puts "The sorted array is "
for val in A
  puts val
end
puts "Serching the elements from an array"
puts "Enter the number to be searched"
```



```
key=gets
if A.include?(key)
  puts "The element #{key} is present in the list"
else
  puts "The element #{key} is not present in the list"
end
```

Output(Run1)

```
Enter the elements in the array
30
10
50
40
20
The sorted array is
10
20
30
40
50
Serching the elements from an array
Enter the number to be searched
40
The element 40
is present in the list
```

(Run1)

```
Enter the elements in the array
30
20
10
50
40
The sorted array is
10
20
30
40
50
Serching the elements from an array

Enter the number to be searched
88
The element 88
is not present in the list
```

6.6 Hashes

- A hash is a collection of key-value pairs like this : "Roll" =>10
- Hashes are also called as **associative arrays**.
- The hashes are similar to arrays but the only difference between the two is that,

Hashes	Arrays
The hash uses the string values as a subscript to address the elements. These string values are basically the keys.	The arrays use the integer values as a subscript to address the element.
The elements in hash are not ordered. The arrangement of elements in hash resembles the set .	The elements in array are ordered by subscript. The arrangement of elements in array resembles the list .

- The hash can be created using either the **new** method or by assigning the key-value pair. For example :

Ruby Program [HashDemo1.rb]

```
A = Hash.new( "one" )
puts "\tFirst Method of Hash Creation"
puts "#{A[0]}"
puts "#{A[3]}"
puts "#{A[5]}"
puts "\tSecond Method of Hash Creation"
B=Hash["AAA"=>100,"BBB"=>200,"CCC"=>300]
puts "#{B['AAA']}"
puts "#{B['BBB']}"
puts "#{B['CCC']}"
puts "\t Displaying the keys of the array"
keys=B.keys
puts "#{keys}"
```

Output

```
First Method of Hash Creation
one
one
one
Second Method of Hash Creation
100
200
300
Displaying the keys of the array
["AAA", "BBB", "CCC"]
```

6.7 Methods

- Methods in Ruby are similar to the functions.
- When a block of statements that get repeated many times in the program then the method is used.
- The method is defined outside the class.
- In this section we will discuss how to write the methods in Ruby and call them.

6.7.1 Fundamentals

- The method definition includes **method header** and sequence of statements.
- The **method header** consists of the reserved word **def**.
- **The method name must begin with lower case.**
- The syntax of method definition is as given below -

```
def method_name ( argument list )
  method body
end
or
def method_name
  method body
end
```

- If the method has no parameter then the parenthesis can be omitted.

Ruby Program[MethodDemo.php]

```
def mymethod
  puts " This is the method demo"
end
mymethod
```

Output

```
This is the method demo
```

Program Explanation :

- In above, program, we have defined the method named **mymethod**. The method is called only by its name. The keyword **def** is used to define the method.
- Every method in Ruby returns something. The values can be returned using the keyword **return** If there is no return statement then by default the last sentence of the method is returned. For example :

```
def mymethod1
  i=10
  j=20
end
puts mymethod1
```

The above code will display the output as 20.

- We can make use of the **return** statement to return the values.
- In Ruby we can return more than one values.

For example :

```
def mymethod2
  i=10
  j=20
  return i,j
end
puts mymethod2
```

Output

```
10
20
```

6.7.2 Local Variables

- Local variables are the variables that are created inside the method.
- The scope of the method during the definition of the method.
- The **advantage** of local variables in the method is that if the name of the local variable conflicts with the name of the global variable then the conflict gets resolved automatically by using the local variable.
- The name of the local variable must begin with either the lower case letter or by underscore.
- The lifetime of the local variable is the period of time over which it exists.

6.7.3 Parameters

- There are two types of parameters - **Actual parameter** and the **formal parameter**.
For example :

Ruby Program[testmethod.rb]

```
def testmethod(arg1,arg2)
  puts "The first parameter is #{arg1}"
  puts "The second parameter is #{arg2}"
end
puts testmethod 10,20
```

Output

```
The first parameter is 10
The second parameter is 20
```

- Variable number of parameters can be handled by the method using the asterisk *.
For example :

Ruby Program [testmethod1.rb]

```
def testmethod1(*arg)
  puts "The total number of parameters are: #{arg.length}"
  for i in 0...arg.length
    puts "The parameters are #{arg[i]}"
  end
end
testmethod1 10,20,30
testmethod1 11.1,12.2,13.3,14.4,15.5
```

Output

```
The total number of parameters are: 3
The parameters are 10
The parameters are 20
The parameters are 30
The total number of parameters are: 5
The parameters are 11.1
The parameters are 12.2
The parameters are 13.3
The parameters are 14.4
The parameters are 15.5
```

- The call to the method must have same number of actual parameters as the number of formal parameters.

6.8 Classes

- Ruby is purely an object oriented programming language. Hence **class** is an important factor of this programming language.
- The objects are created from the classes. In other words we can say object is an instance of a class.

6.8.1 Basics of Classes

- Every class contains two things the variables and the functions. The class in the Ruby can be defined as follows -

```
class className
  ...
end
```

- The class name must begin with the capital letter. The keyword **class** can be used to define the class. The class must be terminated by the keyword **end**.

- In Ruby there are two types of variables in class -
 - **Instance Variables** : The instance variables are defined in the method. The instance variables change from object to object. These variables are available across the methods. The syntax of these variables is ,

```
@variable_name
```

- **Class Variables** : The class variables belongs to the class. They are used to define the characteristics of the class. These are available across the objects. The syntax of these variables is,

```
@@variable_name
```

- **Objects** are the instances of the class and these can be created using the method **new**.

For example :

```
Obj1= Myclass.new
```

- There is a method named **initialize** is a constructor of the class which initializes the values of the variables. Using the **def** and the **end** the method initialize can be defined.
- The name of the method must be starting with the lower case letter. It must be ended by the keyword **end**.

Example : Create a class for obtaining the roll number,name and marks of the student. Display the records.

```
class Student
  @@TotalStudents=0
  def initialize(roll, name, marks)
    @stud_roll=roll
    @stud_name=name
    @stud_marks=makes
  end
  def display()
    puts "Roll Number: #@stud_roll"
    puts "name: #@stud_name"
    puts "Marks: #@stud_marks"
  end
  def total()
    @@TotalStudents += 1
    puts "\tTotal number of students: #@@TotalStudents"
  end
end
S1=Student.new("1", "Shilpa", 97.11)
```

Class Variables

Instance Variables

```
S2=Student.new("2", "Rashmi", 89.21)
# Call Methods
S1.display()
S1.total()
puts "-----"
S2.display()
S2.total()
```

Output

```
F:\Ruby-Examples>Ruby classDemo.rb
Roll Number: 1
name: Shilpa
Marks: 97.11
Total number of students: 1
-----
Roll Number: 2
name: Rashmi
Marks: 89.21
Total number of students: 2
```

6.8.2 Access Control

- There are three access control for the methods - **Public**, **private** and **protected**.
- The default access control for the method is **public**.
- The **public** access method means that the method can be called by any code.
- The **protected** access means only objects of defining class and its subclasses may call the method.
- The **private** access means private methods of another object can not be accessed by one object.
- For example

Ruby Program [Test.rb]

```
class Test
  def method1
    puts "This is the first method"
  end
  private
  def method2
    puts "This is second method"
  end
  protected
  def method3
```

```
puts "This is third method"
end
end
obj=Test.new()
obj.method1
```

Output

```
This is the first method
```

Note that if you call the obj.method2 or obj.method3 then we will get the error.

6.8.3 Inheritance

- Inheritance is the most important feature of any object oriented programming language.
- By inheritance the two classes are created one **base class** and another **derived class**.
- Using the object of derived class one can access the methods of itself and the base class both.
- The syntax for declaring the derived class is,

```
class Derived_class_name < Base_class_name
```

For example :

Ruby Program [InheritanceDemo.rb]

```
class A
  def method1
    puts "I am in method1 of class A."
  end
  def method2
    puts "I am in method2 of class A."
  end
end
```

```
class B < A
  def method3
    puts "I am in method3 of class B."
  end
end
obj = B.new
obj.method1
obj.method2
obj.method3
```

Class B is derived from class A

Output

```
I am in method1 of class A.
I am in method2 of class A.
I am in method3 of class B.
```


Note that in above code, using the object of derived class we can access the methods of base class (i.e. method1 and method2) and the derived class (method3) as well.

6.9 Code Blocks and Iterators

- Block consists of collection of code.
- The block is always enclosed within { and }.
- Blocks can be used with methods.
- Using the iterator **each** we can display each element of the array and apply it to block.
- For example :

Ruby Program [BlockDemo.rb]

```
A=[10,20,30,40]
A.each{|num| puts "The number is #{num}"}
```

Output

```
The number is 10
The number is 20
The number is 30
The number is 40
```

- Using the iterator **step** the specific values can be displayed. For Example

Ruby Program [BlockDemo1.rb]

```
10.step(20,2){|num| puts "The number is #{num}"}
```

Starting value ending value Step size

Output

```
The number is 10
The number is 12
The number is 14
The number is 16
The number is 18
The number is 20
```

- Using the **yield** method the block can be invoked.

- A block is always invoked from a function. But the name of the function and the name of the block must be the same in this case.
- For example :

Ruby Program [BlockDemo1.rb]

```
def myfun
```

```
  puts "This statement is in method"
```

```
  yield
```

```
  puts "Returning back to the method"
```

```
end
```

```
myfun {puts "This statement is in block"}
```

Block is called

Block is defined

Output

This statement is in method

This statement is in block

Returning back to the method

6.10 Pattern Matching

6.10.1 Basics of Pattern Matching

- For pattern matching the operator `=~` is used.
- The pattern can be placed within `/`
- For example :
- Open the Interactive Ruby interpreter and type the following commands and observe the results.

```
irb(main):001:0> T="Success"
```

```
=> "Success"
```

```
irb(main):002:0> T=~es/
```

```
=> 4
```

```
irb(main):003:0>
```

- The **split** method is used to separate out the words from the given string. For example following commands on **Interactive Ruby interpreter** will show how to separate out the words from the string using split command.

```
irb(main):005:0> s="hello friends how are you"
=> "hello friends how are you"
irb(main):006:0> w=s.split(/[.]\s*/)
=> ["hello", "friends", "how", "are", "you"]
irb(main):007:0>
```

6.10.2 Remembering Matches

- Ruby allows to **save** the **matched part** of the **pattern** in the variable. Later on we can use this variable to get the desired part of the pattern.
- For example - Open the Interactive Ruby interpreter and type the following command and observe the results -

```
irb(main):001:0> pattern="Anuradha-19999"
=> "Anuradha-19999"
irb(main):002:0> pattern=~ /(\w+)-(\d+)/
=> 0
irb(main):003:0> puts "First string is #{ $1}"
First string is Anuradha
=> nil
irb(main):004:0> puts "Second string is #{ $2}"
Second string is 19999
=> nil
irb(main):005:0>
```

- In above example we have first stored the pattern in a variable say **pattern**. Then this pattern is matched with the regular expression. The `\w` stands for matching the words. The `\d` stands for matching the digits. Thus we get the groups of substrings from the given pattern. Then using `$1, $2, ...` we can retrieve particular part of the regular expression. The `$1` stands for the value of first matched substring, `$2` stands for the value of second matched string.

6.10.3 Substitutions

- There are two methods **sub** and **gsub** which are for searching the desired pattern and replacing them.
- The **sub** method searches the first part of the pattern and replace it.

- For example :

```
irb(main):001:0> mystr="PHP is a fun.Enjoy PHP"
```

```
=> "PHP is a fun.Enjoy PHP"
```

```
irb(main):002:0> mystr.sub(/PHP/,"Ruby")
```

```
=> "Ruby is fun.Enjoy PHP"
```

```
irb(main):003:0>
```

Note that the first occurrence of PHP is replaced by the word Ruby and the second occurrence remains

- The **gsub** method is similar to **sub** but it replaces all the occurrences of matched substring. For example :

```
irb(main):001:0> mystr="PHP is a fun.Enjoy PHP"
```

```
=> "PHP is a fun.Enjoy PHP"
```

```
irb(main):002:0> mystr.gsub(/PHP/,"Ruby")
```

```
=> "Ruby is a fun.Enjoy Ruby"
```

```
irb(main):003:0>
```

Note that the all the occurrences of PHP are replaced by the word Ruby and the second occurrence remains unchanged.

- Using **sub** or **gsub** have the mutator versions and those are **sub!** and **gsub!**. These versions replace the original string also. For example :

```
irb(main):001:0> str="PHP is a fun. I like PHP"
```

```
=> "PHP is a fun. I like PHP"
```

```
irb(main):002:0> str.sub!(/PHP/,"Ruby")
```

```
=> "Ruby is a fun. I like PHP"
```

```
irb(main):003:0> str
```

```
=> "Ruby is a fun. I like PHP"
```

```
irb(main):004:0> str1="Java is OOP.I program in Java"
```

```
=> "Java is OOP.I program in Java"
```

```
irb(main):005:0> str1.gsub!(/Java/,"Ruby");
```

```
irb(main):006:0> str1
```

```
=> "Ruby is OOP.I program in Ruby"
```

Part II : Introduction to Rails

6.11 Overview of Rails

- Rails is a development framework for web - based applications. As Rails are closely associated with Ruby it is often referred as **Ruby on Rails** or simply **RoR**.
- Rails was developed by **David Heinemeier Hansson** in early 2000 and was released in July 2004.

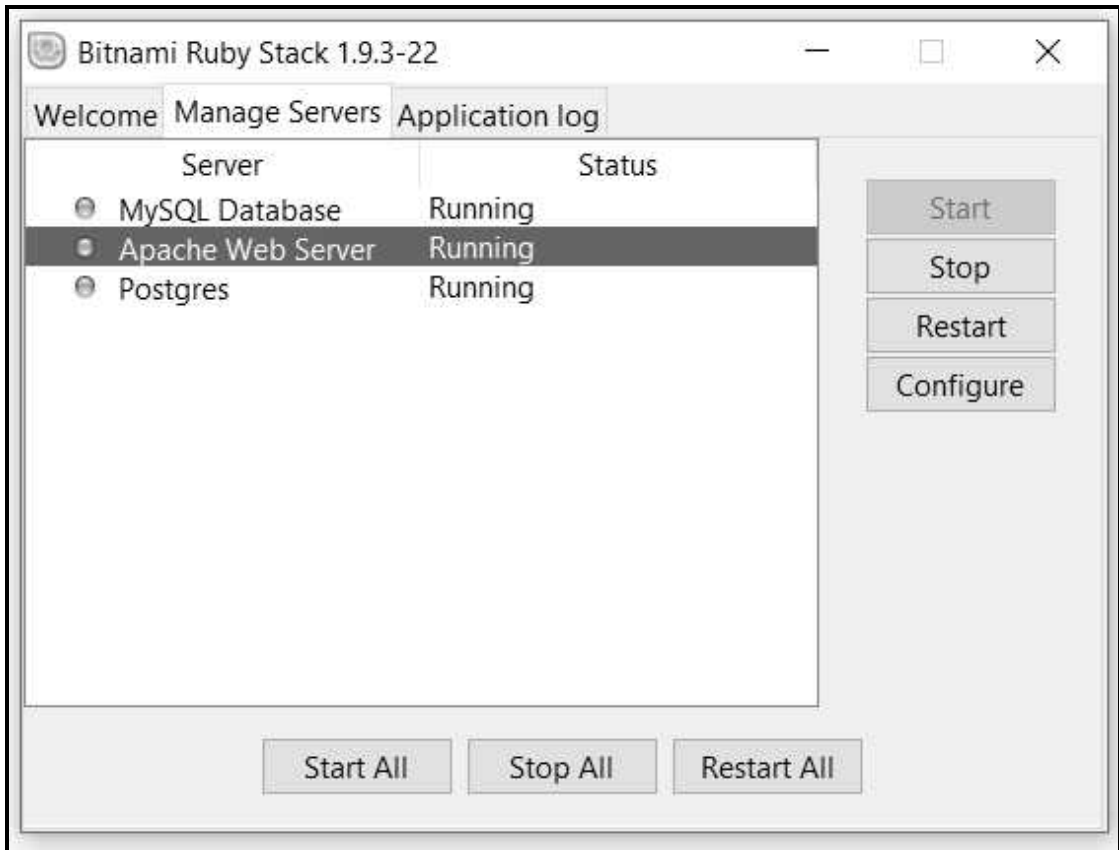
- The web application can be developed ten times faster with Rails than with a typical Java framework.
- Rails is based on **MVC architecture** - i.e. Model View Controller.
- The MVC architecture clearly separates the logical and physical parts of the applications in three parts - Model, View and Controller.
- The **Model** part is for data and constraints on data. The **View** part prepares and presents the result or output. The **Controller** controls the application.
- The most significant characteristic of Rails is its approach to connecting object oriented software with relational database. But both the object oriented software and relational databases are not compatible to each other. Hence Rails make use of **Object-Relational Mapping (ORM) approach**. By this each relational database table is simply mapped to a class. For example if the database has a table named **Student** then it will be mapped to class **Student**. Thus ORM maps **tables to classes**, **rows to objects** and **columns to field of objects**. The ORM of rails is called **activeRecord**.
- The **View** and **Controller** part of MVC are supported by **ActionPack** components of Rails. The view documents are generated for three categories of web documents XHTML, XML and JavaScript and the Controller part of MVC interacts with data model, user and the view.
- Rails can be more commonly used with Ajax. It makes use of the JavaScript framework named prototype to support Ajax and interaction with JavaScript model.
- Rails uses MVC architecture, consists of designing and building three parts of an MVC system.

6.12 Document Requests

- Rails is a software development **framework**. The software development framework consists of library of components that provide commonly needed services to an application.
- The easiest method of downloading the and installing Rails is to make use of a complete software system such as **InstantRails** or **Ruby Stack**.
- The Ruby Stack can be downloaded and installed from <http://bitnami.com/stack/ruby>. It can be downloaded on Windows/ Linux or MacOS platform. It consists of Ruby, Rails, Apache, MySQL, SQLite, Git, PHP and

phpMyAdmin in one pack. Installing Ruby Stack is quick and easy. It is a **self-contained** system.

- After installing the Ruby Stack we can start the Apache Server and MYSQL Server using the **Managers**. The screenshot is as follows.



6.12.1 Static Documents - Creation of First Application in Rails

In this section, we will see the step by step method of building the first application in Rails. The **Ruby Stack** is installed on my machine at `C:\BitNami\rubystack-1.9.3-22`.

Step 1 : Go to the **Projects** directory within the Ruby Stack's directory. Create a folder of any name for storing various applications. I have created a folder named **MyProject1**. This is the base directory in which the developer can add new Rails applications. Now we will create another subdirectory say **test1** by issuing the following command

```
> rails new test1
```

Following screenshot illustrates it.

```
C:\Windows\System32\cmd.exe

C:\Bitnami\rubystack-1.9.3-22\projects\MyProject1>rails new test1
create
create  README.rdoc
create  Rakefile
create  config.ru
create  .gitignore
create  Gemfile
create  app
create  app/assets/images/rails.png
create  app/assets/javascripts/application.js
create  app/assets/stylesheets/application.css
create  app/controllers/application_controller.rb
create  app/helpers/application_helper.rb
```

- Rails respond to this command by generating number of files and directories. Out of these directories the most important directory from the application point of view is **app**. It has four subdirectories namely **models**, **views**, **controllers** and **helpers**. The **models**, **views** and **controllers** are those directories that will support MVC of Rails.
- For the sake of understanding just look at the directory structure as given below -

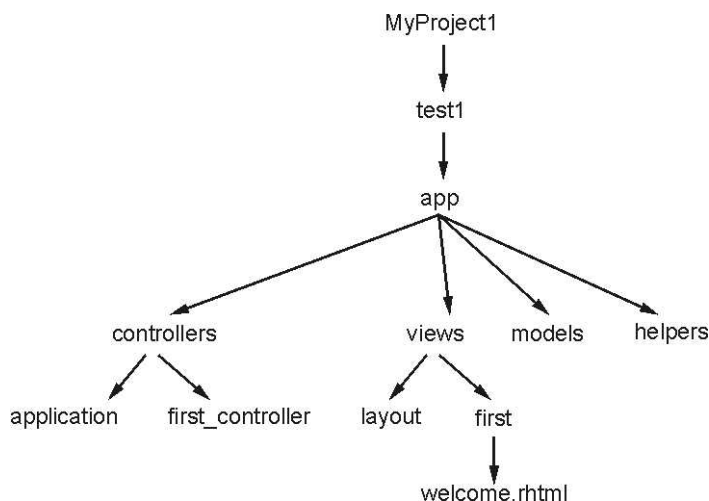
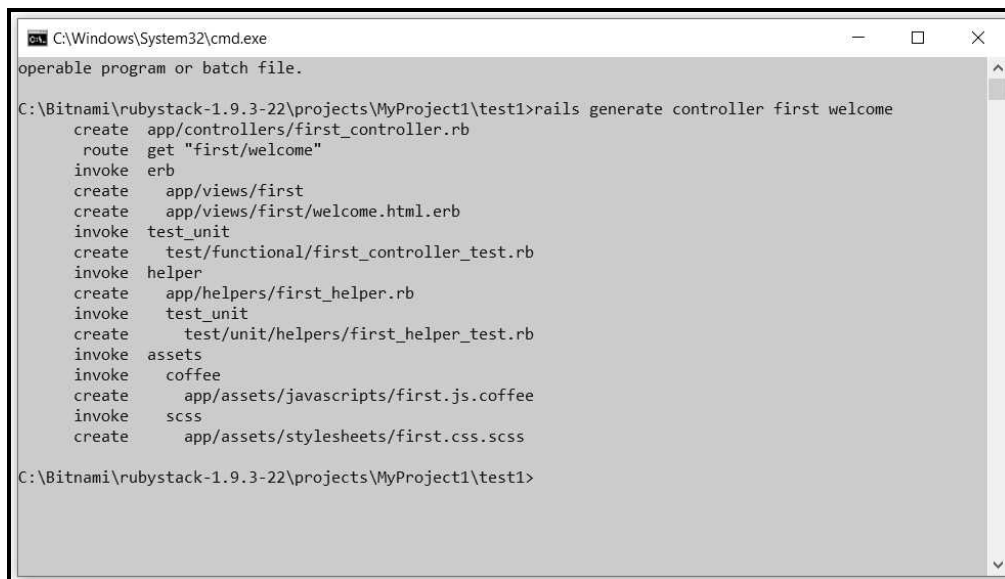


Fig. 6.12.1 Directory structure for the test1 application

Step 2 : This directory contains several important Ruby scripts that perform the services. One of the service is **generate**. This service is used to create or generate the controller methods. The two parameters are passed to this service - first is **controller** and second is the **name chosen for the controller**. The **controller** indicates that we want the **controller class** to be built. The command will be issued from the **test1** directory. The command will be as follows -

```
> rails generate controller first welcome
```

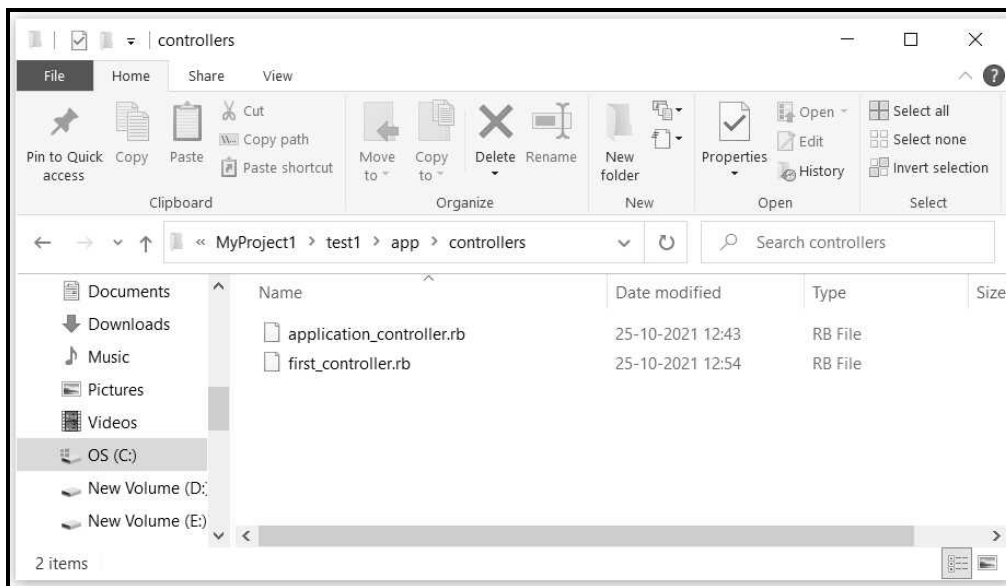


```
C:\Windows\System32\cmd.exe
operable program or batch file.

C:\Bitnami\rubystack-1.9.3-22\projects\MyProject1\test1>rails generate controller first welcome
create    app/controllers/first_controller.rb
route    get "first/welcome"
invoke   erb
create    app/views/first
create    app/views/first/welcome.html.erb
invoke   test_unit
create    test/functional/first_controller_test.rb
invoke   helper
create    app/helpers/first_helper.rb
invoke   test_unit
create    test/unit/helpers/first_helper_test.rb
invoke   assets
invoke   coffee
create    app/assets/javascripts/first.js.coffee
invoke   scss
create    app/assets/stylesheets/first.css.scss

C:\Bitnami\rubystack-1.9.3-22\projects\MyProject1\test1>
```

- Due to the above command the **FirstController** class will be generated in **first_controller.rb** at location **app/controllers/**
- There are now two Ruby classes **application_controller.rb** and **first_controller.rb**.



The **first_controller.rb** can be opened up in Notepad/Wordpad . It contains -

```
class FirstController < ApplicationController
end
```

Here **FirstController** is an empty class and is inherited from **ApplicationController** class.

These Controllers are useful for producing the HTTP responses. To respond the request there must be some **actionmethod**. This actionmethod simply indicate the document as a response. The **first_controller.rb** will be as given below in appearance -

```
class FirstController < ApplicationController
  def welcome
  end
end
```

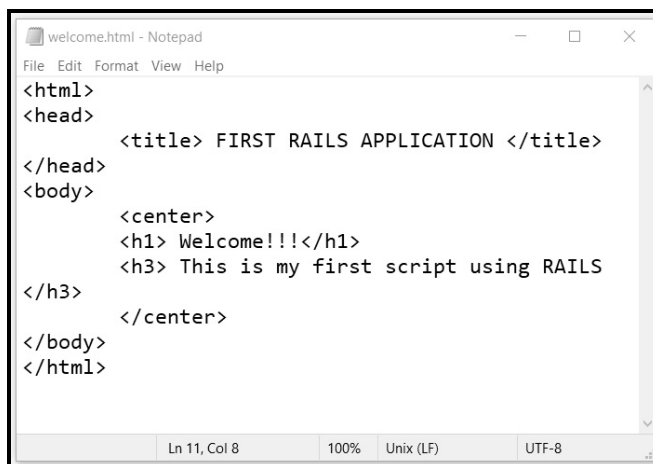
Step 3 : In above step the welcome is the action method name. It is the response to the URL `http://localhost:3000/first/welcome`.

This is actually the view document. The view document is called as **template**. These templates will be placed in **app/views** directory. The template will be written as an XHTML file. The extension to this file will be **.rhtml (or it can be html)**. The Ruby code can be inserted in this file. Hence the extension is derived from the word Ruby HTML. For our first application we can write the template as follows[Note that there is no Ruby code embedded in the following rhtml file, it simply makes use of HTML tags].

welcome.rhtml

```
<html>
<head>
<title> FIRST RAILS PROGRAM</title>
</head>
<body>
<center>
<h1>Welcome!!!</h1>
<h3>This is my First Program using Rails </h3>
</center>
</body>
</html>
```

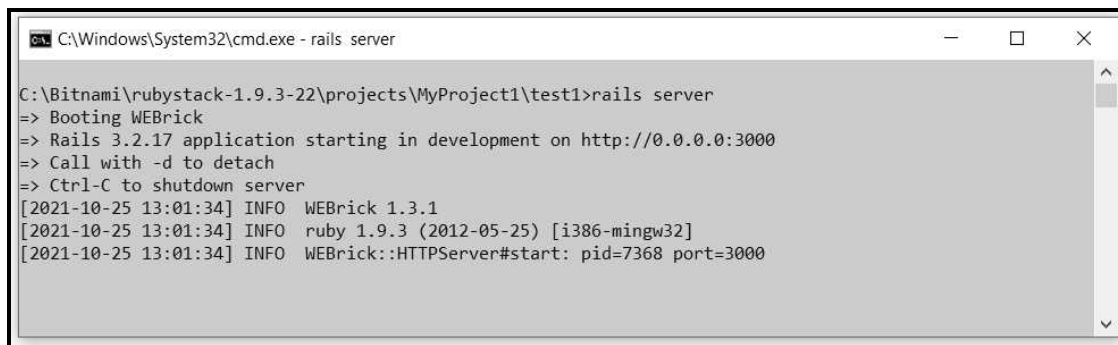
It can be illustrated as –



Step 4 : Now we will start the server for running the above created static document application by issuing the following command

```
> rails server
```

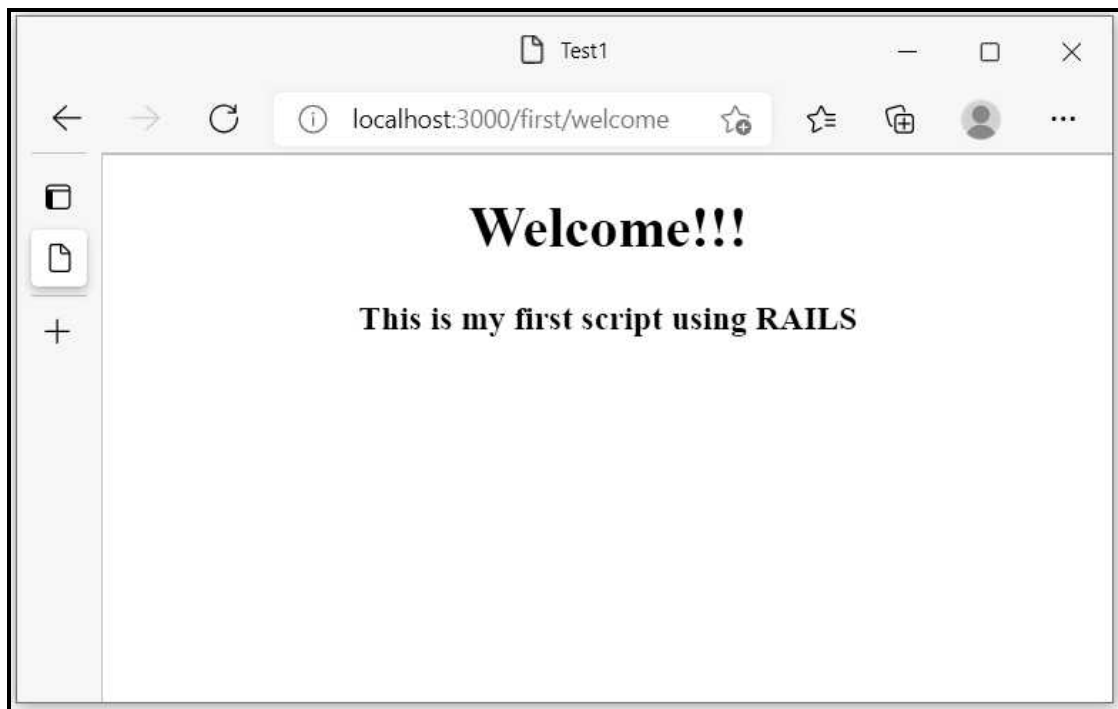
The following screenshot illustrates this command.



```
C:\Windows\System32\cmd.exe - rails server

C:\Bitnami\rubystack-1.9.3-22\projects\MyProject1\test1>rails server
=> Booting WEBrick
=> Rails 3.2.17 application starting in development on http://0.0.0.0:3000
=> Call with -d to detach
=> Ctrl-C to shutdown server
[2021-10-25 13:01:34] INFO WEBrick 1.3.1
[2021-10-25 13:01:34] INFO ruby 1.9.3 (2012-05-25) [i386-mingw32]
[2021-10-25 13:01:34] INFO WEBrick::HTTPServer#start: pid=7368 port=3000
```

Now open suitable web browser and type the URL <http://localhost:3000/first/welcome> on the address bar. [Note that the URL specified during the installation is **localhost** and port is 3000]



6.12.2 Dynamic Documents

- Dynamic document is a kind of web document in which the contents are changing more often. For example if your web page is displaying the current date and time or stock market position or Cricket match score then that document is said to be a dynamic document.
- In this section we will build an application in which the dynamic web document is created for displaying the current date and time. The implementation of such web document is possible by embedding some Ruby code within it.
- The **Ruby code** can be **embedded** within the template file by using `<%` and `%>` markers.
- If the Ruby code produces some result then the result can be inserted within the template document by `<%=` and `%>` marker.
- Ruby's **Time.now** is used to display the current time. In earlier section we have seen how to display the static web document. We will follow steps 1 and 2 from the above section. The template document created in step 3 is slightly changed. It is as follows -

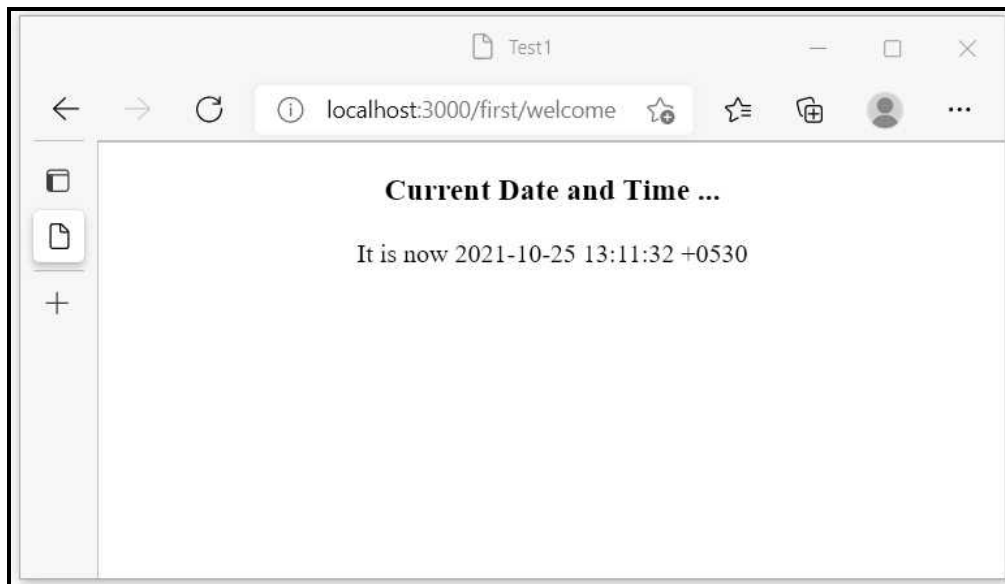
welcome.html

```
<html>
<head>
<title> FIRST RAILS PROGRAM</title>
</head>
<body>
<center>
<h3>Current Date and Time ... </h3>
<p> It is now <%= t = Time.now %> </p>
</center>
</body>
</html>
```

- Now start the server by going into the test1 directory by the command -

```
> Rails server
```

Open the suitable web browser and get the output as follows -



6.13 Processing Forms

Form is a useful tool used in building the GUI using which the user can interact with the system.

We will follow the following steps in processing form using Rails.

Step 1 : We have already created a folder named **MyProject1** in the **Projects** directory of **Ruby Stack**. Now go to the **MyProject1** directory and create a **sampleForm** subdirectory using the following command by using the command prompt -

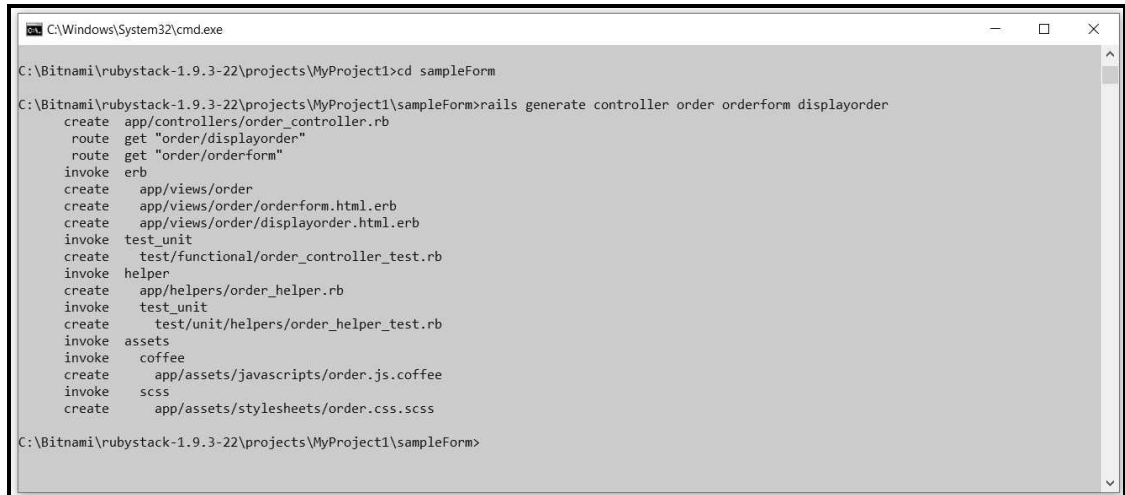
```
C:\Windows\System32\cmd.exe

C:\Bitnami\rubystack-1.9.3-22\projects\MyProject1>rails new sampleForm
create
create  README.rdoc
create  Rakefile
create  config.ru
create  .gitignore
create  Gemfile
create  app
create  app/assets/images/rails.png
create  app/assets/javascripts/application.js
create  app/assets/stylesheets/application.css
create  app/controllers/application_controller.rb
create  app/helpers/application_helper.rb
create  app/mailers
create  app/models
create  app/views/layouts/application.html.erb
create  app/mailers/.gitkeep
create  app/models/.gitkeep
```

Now the **sampleForm** will have all those necessary directories such as **app**, **config**, **script** and so on generated within it.

Step 2 : Now go to the **sampleForm** directory and issue the following command on the command prompt for generating controller.

>rails generate controller order orderform displayorder



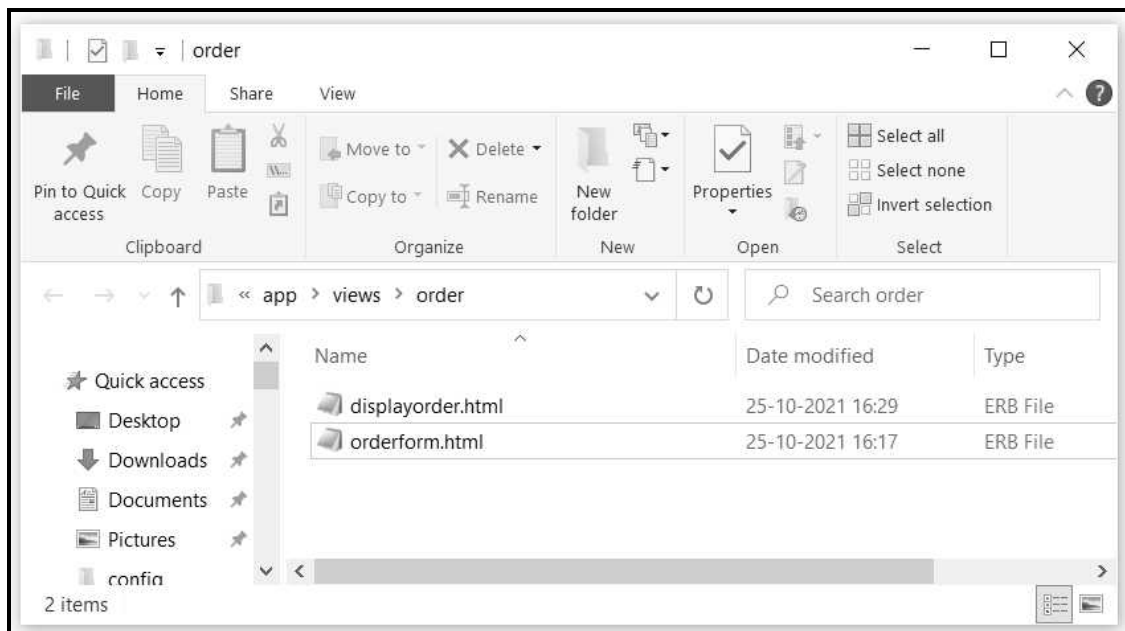
```
C:\Windows\System32\cmd.exe

C:\Bitnami\rubystack-1.9.3-22\projects\MyProject1>cd sampleForm

C:\Bitnami\rubystack-1.9.3-22\projects\MyProject1\sampleForm>rails generate controller order orderform displayorder
create    app/controllers/order_controller.rb
route    get "order/displayorder"
route    get "order/orderform"
invoke   erb
create   app/views/order
create   app/views/order/orderform.html.erb
create   app/views/order/displayorder.html.erb
invoke   test_unit
create   test/functional/order_controller_test.rb
invoke   helper
create   app/helpers/order_helper.rb
invoke   test_unit
create   test/unit/helpers/order_helper_test.rb
invoke   assets
invoke   coffee
create   app/assets/javascripts/order.js.coffee
invoke   scss
create   app/assets/stylesheets/order.css.scss

C:\Bitnami\rubystack-1.9.3-22\projects\MyProject1\sampleForm>
```

This will generate **order_controller.rb** file. Also create **orderform.html** and **displayorder.html** files in the **order** sub-directory.



Step 3 : The code for orderform.html file is as follows -

orderform.html

```
<html>
  <head>
    <title> Submitted Order</title>
  </head>
  <body>
    <h3> User Name: <%= @UserName %> </h3>
    <table border="1">
      <tr>
        <td><b>Product Name</b></td>
        <td><b>Price(Rs.)</b></td>
        <td><b>Quantity</b></td>
        <td><b>Cost</b></td>
      </tr>
      <tr>
        <td>Apples</td>
        <td>10 </td>
        <td><%= @qty1 %></td>
        <td><%= @apple_cost %></td>
      </tr>
      <tr>
        <td>Oranges</td>
        <td>7.50 </td>
        <td><%= @qty2 %></td>
        <td><%= @orange_cost %></td>
      </tr>
    </table>
    <br/>
    <h2> Total Cost = <%= @total_cost %> </h2>
  </body>
</html>
```

Step 4 : Edit the controller file **order_controller.rb** as follows -

order_controller.rb

```
class OrderController < ApplicationController
```

```
  def orderform
```

Invokes orderform.html

```
  end
```

```
  #displayorder method - it fetches the data from form and compute
```

```
  #total cost
```

```
  def displayorder
```

Invokes displayorder.html

```
    @UserName =params[:UserName]
```

```
    @qty1 = params[:qty1].to_i
```

```
    @qty2 = params[:qty2].to_i
```

```
  #Computing the cost of each item
```

```
  @apple_cost = 10 * @qty1
```

```
  @orange_cost = 7.50 * @qty2
```

```
  #computing the total cost of purchased items
```

```
  @total_cost = @apple_cost+@orange_cost
```

Reading the input from the orderform.html and preparing the computed cost variables for displaying the result.

```
  #Converting the values into strings with two digits
```

```
  @total_cost = sprintf("%.2f",@total_cost)
```

```
  @apple_cost = sprintf("%.2f",@apple_cost)
```

```
  @orange_cost = sprintf("%.2f",@orange_cost)
```

```
  end
```

```
end
```

- Using the **params** symbol the parameters on the input rhtml file can be invoked. The **.to_i** method is used to convert the string to integers.
- The **sprintf** function of C can be used to format the numbers in two decimal places. This is done for displaying the floating point data in formatted manner.
- Save this file and close it. By the above code we have actually written the response methods namely **orderform** and **displayorder**. The **orderform.html** and **displayorder.html** files will be invoked as a response to these action methods. Out of these the **orderform.html** is already created. Now we will created **displayorder.html** web document.

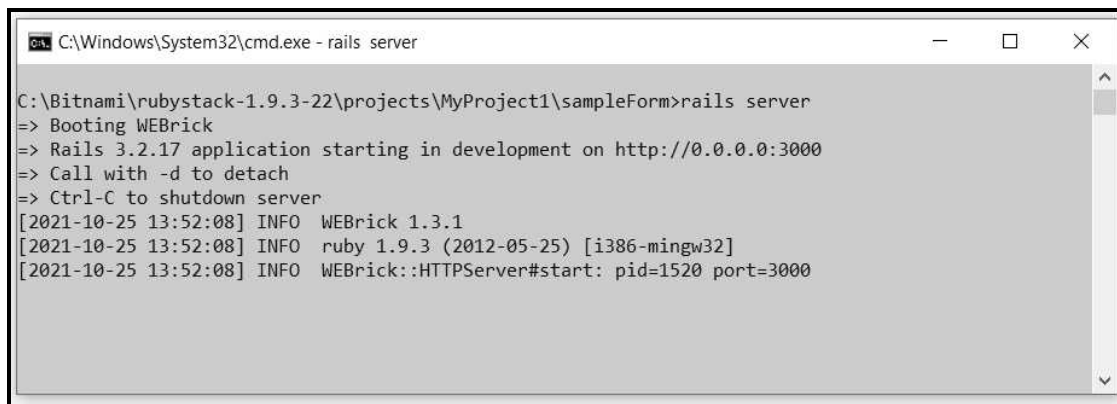
Step 5 : Open `app/views/order` and create a file named `displayorder.html`. It is as follows -

displayorder.html

```
<html>
  <head>
    <title> Submitted Order</title>
  </head>
  <body>
    <h3> User Name: <%= @UserName %> </h3>
    <table border="1">
      <tr>
        <td><b>Product Name</b></td>
        <td><b>Price(Rs.)</b></td>
        <td><b>Quantity</b></td>
        <td><b>Cost</b></td>
      </tr>
      <tr>
        <td>Apples</td>
        <td>10 </td>
        <td><%= @qty1 %></td>
        <td><%= @apple_cost %></td>
      </tr>
      <tr>
        <td>Oranges</td>
        <td>7.50 </td>
        <td><%= @qty2 %></td>
        <td><%= @orange_cost %></td>
      </tr>
    </table>
    <br/>
    <h2> Total Cost = <%= @total_cost %> </h2>
  </body>
</html>
```

Step 6 : Now open command prompt and start the server using the command.

```
> rails server
```

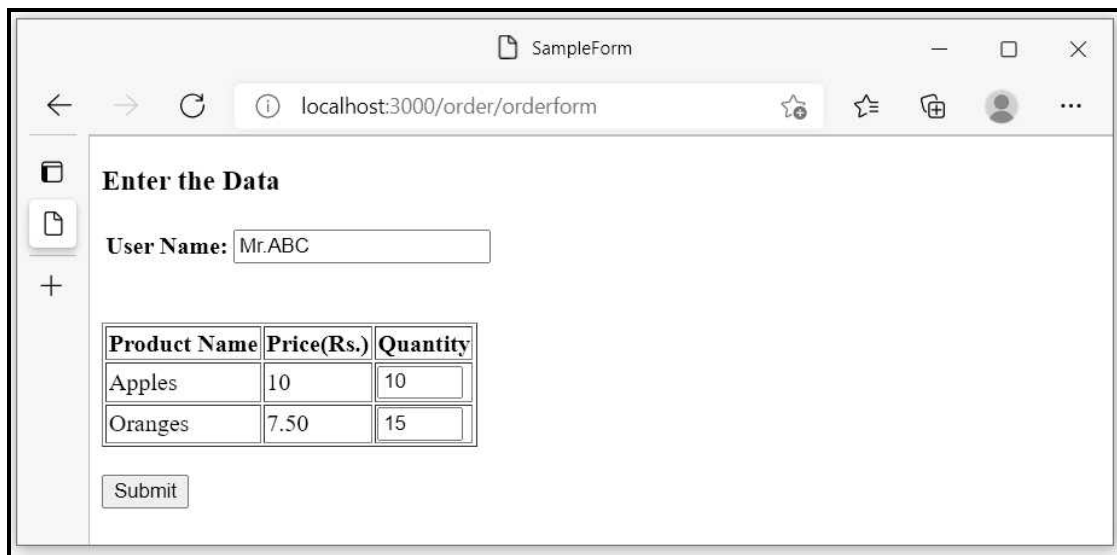
```
C:\Windows\System32\cmd.exe - rails server

C:\Bitnami\rubystack-1.9.3-22\projects\MyProject1\sampleForm>rails server
=> Booting WEBrick
=> Rails 3.2.17 application starting in development on http://0.0.0.0:3000
=> Call with -d to detach
=> Ctrl-C to shutdown server
[2021-10-25 13:52:08] INFO  WEBrick 1.3.1
[2021-10-25 13:52:08] INFO  ruby 1.9.3 (2012-05-25) [i386-mingw32]
[2021-10-25 13:52:08] INFO  WEBrick::HTTPServer#start: pid=1520 port=3000
```

Then open the suitable web browser and type the following URL in the address bar.

<http://localhost:3000/order/orderform>

Enter the data and click the **Submit** button on that form. You will get following output.



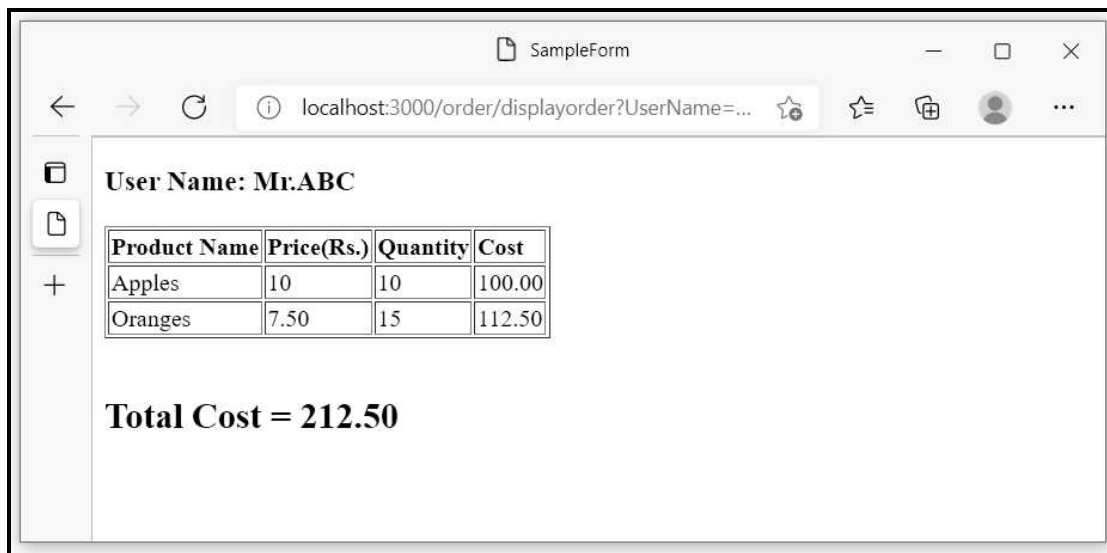
SampleForm

localhost:3000/order/orderform

Enter the Data

User Name:

Product Name	Price(Rs.)	Quantity
Apples	10	10
Oranges	7.50	15



6.14 Rails Applications and Databases

Step 1 : For creating a database application create a folder by some suitable name in Ruby's directory. I have created a folder named **dbdemo** by following command

```
> rails new dbdemo
```

The screenshot is as follows -

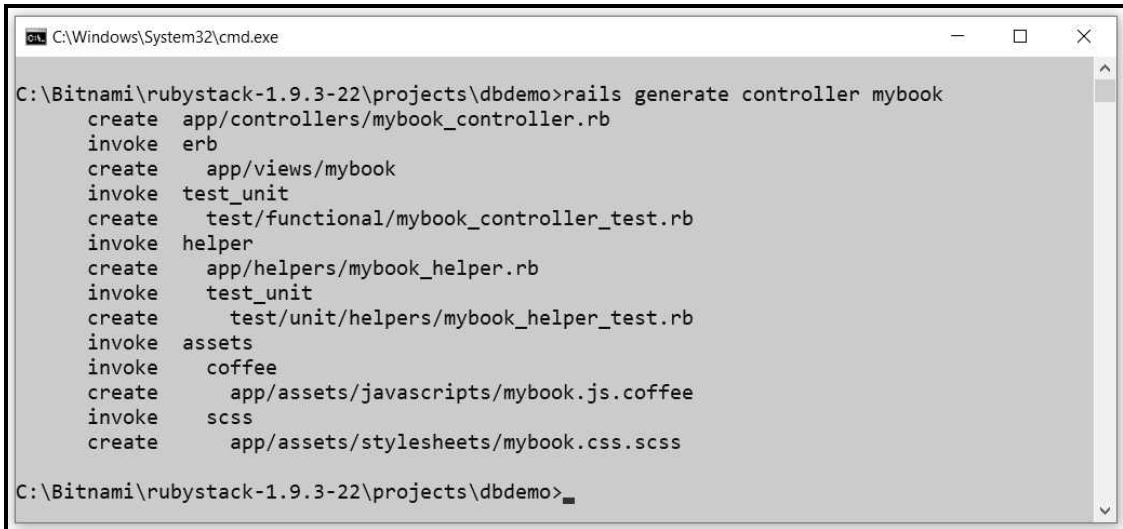
```
C:\Windows\System32\cmd.exe

C:\Bitnami\rubystack-1.9.3-22\projects>rails new dbdemo
create
create  README.rdoc
create  Rakefile
create  config.ru
create  .gitignore
create  Gemfile
create  app
create  app/assets/images/rails.png
create  app/assets/javascripts/application.js
create  app/assets/stylesheets/application.css
create  app/controllers/application_controller.rb
create  app/helpers/application_helper.rb
create  app/mailers
create  app/models
create  app/views/layouts/application.html.erb
create  app/mailers/.gitkeep
```

By this command a directory **dbdemo** will be created with several useful subdirectories such as **app**, **config**, **doc**, **db**, **script** and so on.

Step 2 : Now generate the **controller** by the following command in **dbdemo** directory

```
> rails generate controller mybook
```



```
C:\Windows\System32\cmd.exe

C:\Bitnami\rubystack-1.9.3-22\projects\dbdemo>rails generate controller mybook
   create  app/controllers/mybook_controller.rb
   invoke  erb
   create  app/views/mybook
   invoke  test_unit
   create  test/functional/mybook_controller_test.rb
   invoke  helper
   create  app/helpers/mybook_helper.rb
   invoke  test_unit
   create  test/unit/helpers/mybook_helper_test.rb
   invoke  assets
   invoke  coffee
   create  app/assets/javascripts/mybook.js.coffee
   invoke  scss
   create  app/assets/stylesheets/mybook.css.scss

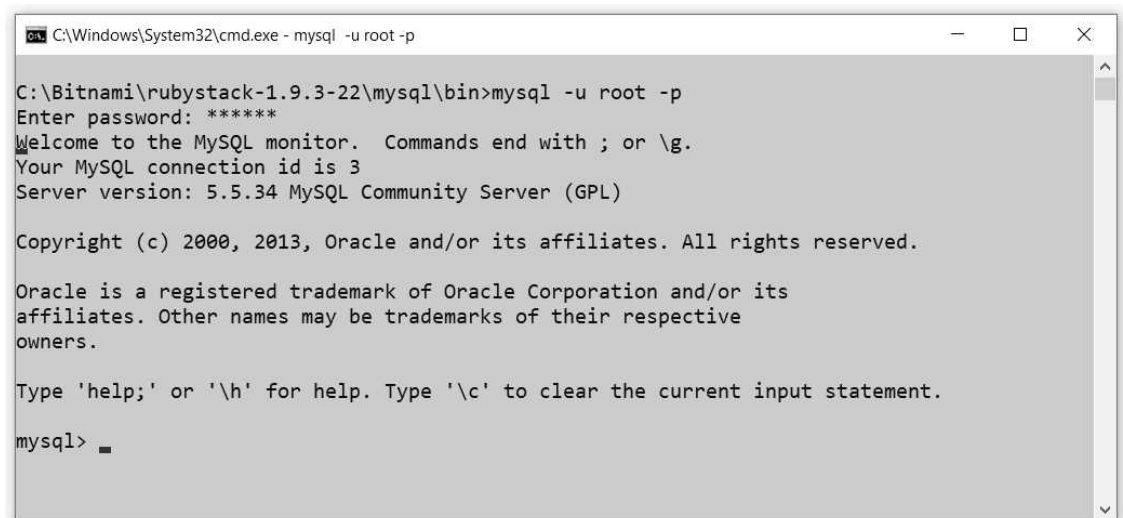
C:\Bitnami\rubystack-1.9.3-22\projects\dbdemo>
```

So a controller named **mybookController** will be created within the file **mybook_controller.rb** of **controller** directory.

Step 3 : Now we will create a database in the MySQL by following command

```
>mysql -u root -p;
```

It will prompt for password if any. Then the **mysql** prompt will appear on the prompt window.



```
C:\Windows\System32\cmd.exe - mysql -u root -p

C:\Bitnami\rubystack-1.9.3-22\mysql\bin>mysql -u root -p
Enter password: *****
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 3
Server version: 5.5.34 MySQL Community Server (GPL)

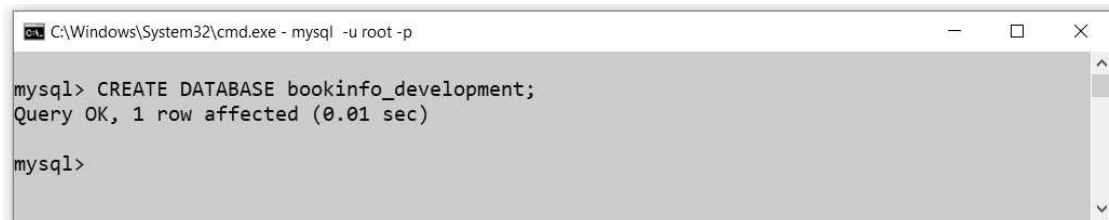
Copyright (c) 2000, 2013, Oracle and/or its affiliates. All rights reserved.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> _
```

Step 4 : Now create a database named **bookinfo_development** using CREATE DATABASE command. Following screenshot illustrates it.

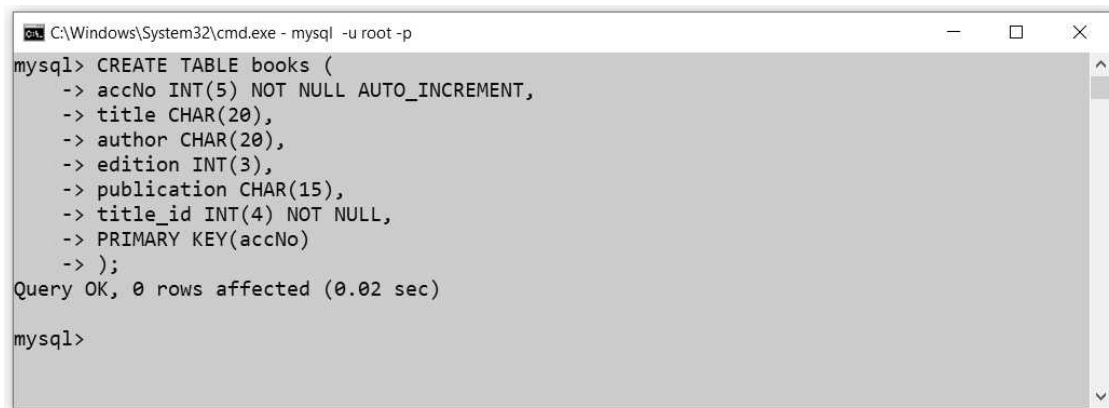


```
C:\Windows\System32\cmd.exe - mysql -u root -p

mysql> CREATE DATABASE bookinfo_development;
Query OK, 1 row affected (0.01 sec)

mysql>
```

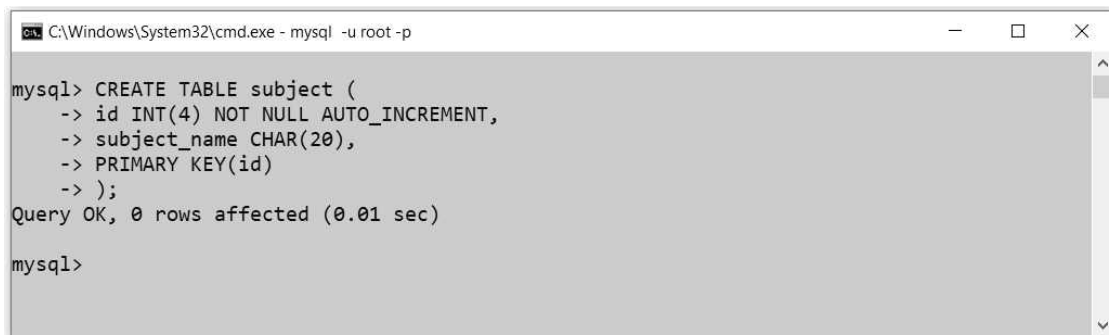
Then create tables **books** and **subject**. Following screenshot illustrates the creation commands.



```
C:\Windows\System32\cmd.exe - mysql -u root -p

mysql> CREATE TABLE books (
-> accNo INT(5) NOT NULL AUTO_INCREMENT,
-> title CHAR(20),
-> author CHAR(20),
-> edition INT(3),
-> publication CHAR(15),
-> title_id INT(4) NOT NULL,
-> PRIMARY KEY(accNo)
-> );
Query OK, 0 rows affected (0.02 sec)

mysql>
```



```
C:\Windows\System32\cmd.exe - mysql -u root -p

mysql> CREATE TABLE subject (
-> id INT(4) NOT NULL AUTO_INCREMENT,
-> subject_name CHAR(20),
-> PRIMARY KEY(id)
-> );
Query OK, 0 rows affected (0.01 sec)

mysql>
```

Step 5 : Now we will generate the model files that are required for this application. Use following command for this.

```
C:\Windows\System32\cmd.exe

C:\Bitnami\rubystack-1.9.3-22\projects\dbdemo>rails generate model book
  invoke  active_record
  create  db/migrate/20211026072342_create_books.rb
  create  app/models/book.rb
  invoke  test_unit
  create  test/unit/book_test.rb
  create  test/fixtures/books.yml

C:\Bitnami\rubystack-1.9.3-22\projects\dbdemo>rails generate model subject
  invoke  active_record
  create  db/migrate/20211026072358_create_subjects.rb
  create  app/models/subject.rb
  invoke  test_unit
  create  test/unit/subject_test.rb
  create  test/fixtures/subjects.yml

C:\Bitnami\rubystack-1.9.3-22\projects\dbdemo>
```

These command will generate the files namely book.rb and subject.rb in app/model directory.

Open these files using the text editor and edit them as follows -

book.rb

```
class Book < ActiveRecord::Base
  belongs_to :subject
end
```

subject.rb

```
class Subject < ActiveRecord::Base
  has_many :books
end
```

The **has_many** and **belongs_to** are the relationships used in MySQL for specifying the classes associated with the database tables. When these relationships are provided then Rails generate the capabilities to manipulate the given relationship.

Step 6 : Now insert the data into tables **books** and **subject**. Following commands illustrates the insertion of data.

```
C:\Windows\System32\cmd.exe - mysql -u root -p

mysql> INSERT INTO books VALUES
-> (10,"Quantum Physics","AAA",2,"XXX",5),
-> (20,"Organic Chemistry","BBB",1,"ZZZ",3),
-> (30,"Mental Maths","CCC",1,"YYY",2);
Query OK, 3 rows affected (0.01 sec)
Records: 3  Duplicates: 0  Warnings: 0
```

```
C:\Windows\System32\cmd.exe - mysql -u root -p
mysql> INSERT INTO subject VALUES
-> (5,"Physics"),
-> (2,"Chemistry"),
-> (3,"Mathematics");
Query OK, 3 rows affected (0.00 sec)
Records: 3 Duplicates: 0 Warnings: 0
```

Step 7 : Now open the controller file which is present in the controller folder. Edit it as follows -

mybook_controller.rb

```
class MybookController < ApplicationController
  def welcome
  end
  def show
    @titlename = params[:title]
    @bookarr = book.find(:all, :conditions => ["title = ?", @titlename])
    @subarr = subject.find(:all)
  end
end
```

Step 8 : The **welcome.rhtml** file can be stored in **app / views** directory. It will be as follows -

```
<html>
<head><title> RAILS-MYSQL DEMO </title> </head>
<body>
<form method="post" action="show">
  <input type="text" name="title" size="20"/>
  <input button="submit" name="Display" />
</form>
</body>
</html>
```

The **show.rhtml** file can be stored in **app / views** directory. It will be as follows -

```
<html>
  <head>
    <title> Book Display</title>
  </head>
  <body>
    <table border="1">
      <tr>
        <td><b>AccessNo</b></td>
        <td><b>Title</b></td>
        <td><b>Author</b></td>
        <td><b>Edition</b></td>
```

```

<td><b>Publication</b></td></td>
<td><b>Subject</b></td>

</tr>
<% @bookarr.each do |b|
  @accno=b.accNo
  @title=b.tilte
  @author=b.author
  @edition=b.edition
  @pub=b.publication
  @subject_row=@subarr[b.title_id]
  @subjectname=@subject_row.subject_name %>
<tr>
  <td><b><%= accno %></b></td>
  <td><b><%= title %></b></td>
  <td><b><%= author %></b></td>
  <td><b><%= edition %></b></td>
  <td><b><%= pub %></b></td>
  <td><b><%= subjectname %></b></td>
</tr>
<% end %>
</table>
</body>
<html>

```

6.15 Layouts

- The **app/View** directory of each application has two directories, one is for the controller and another is named as **layouts**.
- A layout defines the surroundings of an HTML page. In layout directory you can place a file for common look and feel of your final output. For that purpose layout template is created.
- Rails must be informed about the name of the layout document by placing the **layout** command in the **ApplicationController** class of the application.
- In the **controller** directory of **app** there is a file named **application_controller.rb** in this file the following code can be placed -

```

class ApplicationController < ActionController::Base
  layout "mainlayout"
end

```

- Now if we add **mainlayout.rhtml** document to layout directory then the basic layout of the application can be defined.

6.16 Rails with Ajax

- AJAX stands for Asynchronous JavaScript and XML. It allows client side changes without reloading the page.
- When you type **http://localhost:3000** into your browser's address bar and hit 'Go', the browser (your 'client') makes a request to the server. It parses the response, then fetches all associated assets, like JavaScript files, stylesheets and images.
- It then assembles the page. If you click a link, it does the same process : Fetch the page, fetch the assets, put it all together, show you the results. This is called the 'request response cycle'.
- JavaScript can also make requests to the server, and parse the response. It also has the ability to **update information** on the page. Combining these two powers, a JavaScript writer can make a web page that can **update just parts of itself**, without needing to get the full page data from the server. This is a powerful technique is called as Ajax.
- For example -

```
fetch("/index")
  .then((data) => data.text())
  .then((html) => {
    const results = document.querySelector("#compute");
    results.insertAdjacentHTML("before_exit", data);
  });
```

- This code fetches data from "/index" and then appends the result to the element with an id of **compute**.
- Rails provides quite a bit of built-in support for building web pages with this technique. Rails uses a technique called "Unobtrusive JavaScript" to handle attaching JavaScript to the DOM. This is a technique by which we can mix JavaScript code into HTML. With this, we can easily add behavior to any link by adding data attribute.

Part III : EJB

6.17 Introduction to EJB

SPPU : May-18, 19, Dec.-18, Marks 8

- J2EE application container contains the components that can be used by the clients for executing the business logic. These components are known as Enterprise Java Beans (EJB).
- EJB mainly contains the business logic and business data.

- Using the technology like EJB the server side, distributed Java components can be built.
- EJBs provide a core set of system services to Java component. **EJB components** always lie in some container which is called as **EJB container**.
- EJB component is an EJB class. It is a Java class written by EJB developer and this class implements business logic.
- EJB component is similar to traditional Java beans but there are some difference between the two. These differences are as given below -

Java Beans	EJB
The java bean components can be visible or invisible to the user.	The EJB components are always invisible to the user.
The Java beans normally run locally.	EJB are distributed components and always exist on the servers.
The Java beans can be used as Activex Control.	EJBs can't be used as Activex Control.
The Java beans can be described with properties, BeanInfo classes and Customizers .	EJBs can be described using DeploymentDescriptor .

6.17.1 Types of EJB

- Following are the types of EJB.

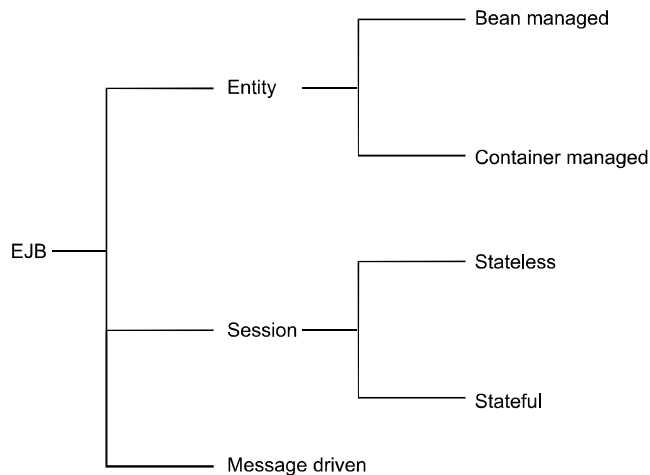


Fig. 6.17.1 Types of EJB

Entity Bean

- Entity beans are used for modelling the business concepts. Therefore entity beans represent the data. For example entity bean can represent the bank account or a customer. The **nouns** that are used in the problem description refer to the entity beans. Basically the entity beans represent the **real world objects**.
- Such beans then act as a **persistent record** in the database.

Session Bean

- Session beans are used for managing processes or tasks. For example *Compute net profit* or *transaction* or *browsing for some information on the web browser*. Hence session beans are used for managing activities. Basically session beans represent the **activities**. Hence session beans are **transient**, because activities generally exist for a short period. Session bean does not represent anything in the database but it can access the database. Session beans are client specific.

What are the differences between entity bean and session bean ?

Sr. No.	Entity bean	Session bean
1.	These are persistent .	These are transient .
2.	It refers to the records in the database.	These refers to the activities for the database.
3.	These can survive on server crash. (After server gets online these can be reconstructed from the underlying data).	Session objects generally cannot survive on server crashes.
4.	Entity beans are client-independent entities.	Session beans are client-specific entities.

Bean managed Persistent

- The bean managed persistent is a kind of entity bean that is used for saving bean's state.
- The container does not need to make any database call.

Container managed Persistent

- This kind of bean is used to manage the persistence of the entity bean.
- That means EJB container is responsible for saving the bean's state.
- For container managed persistent bean implementation is independent of the data source.

Stateless

- Stateless bean is designed for simple and lightweight components.
- This brings the scalability in server performance. As no state is maintained in this type of bean it is called stateless bean.
- The stateless beans are not tied to any specific client. Hence any available instance of stateless session bean can be used to service a client.

Stateful

- The stateful bean is used to store the internal states of the EJB.
- There is only one stateful session bean per EJB client.
- These types of beans can be saved across client sessions. And we can restore them back.

Message driven Bean

- Message driven is similar to the session bean but it gets activated only when asynchronous message arrives.
- A message-driven bean's instances retain no data or state for a specific client.
- A single message-driven bean can process messages from multiple clients.
- When a message arrives then the EJB container calls the message driven bean's `onMessage` method to process the method.

6.17.2 Benefits of EJB

1. Business to Business (B2B) e-commerce applications make use of EJB.
2. EJB can be used in web-centric as well as web-oriented applications.
3. Use of EJB is more suitable when there is large number of clients involved.
4. The Enterprise Application Integration (EAI) can use EJB for processing and mapping different applications. There, encapsulation of business logic with the data is possible due to EJB.
5. They can be customized at deployment time by editing the deployment descriptor.
6. System-level services, such as transaction management and security, are described separately from the enterprise bean.
7. A client never accesses an enterprise bean directly; the container environment mediates access for the client. This provides component-location transparency.
8. The EJB is designed to be portable across EJB servers provided by different vendors.
9. They can be included in an assembled application without requiring source code changes or recompilation of them.

6.17.3 Architecture

Components of EJB

- The EJB container provides following services to EJB component -
 - Security
 - Transactions
 - Messaging
 - Persistence
 - Distributions
 - Connectivity
- The idea behind developing EJB is to provide a framework for the components that can be plugged into a server. This ultimately helps to increase the functionality of the server.
- EJB is a managed component that can be created, controlled and managed by its container J2EE.
- There can be multiple EJBs in the container J2EE. The container J2EE provides the resources (such as memory and database connections) required by the EJB.
- Thus J2EE maintains a pool of EJB instances. These instances can be assigned to the clients.
- Client makes use of these EJB components and when it is no longer required by the client, these instances can be returned to the pool so that other clients can use these instances. The client who is using the EJB instance does not need to know about its container.

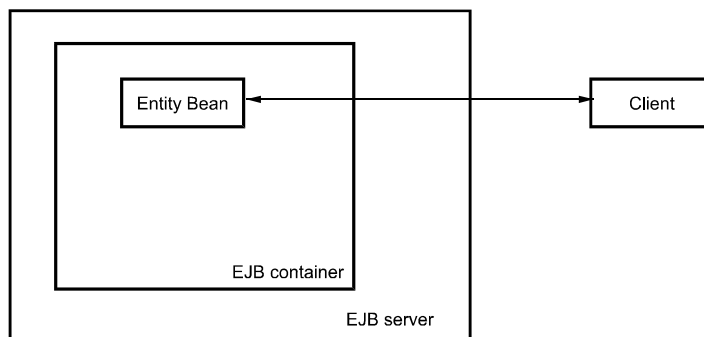


Fig. 6.17.2

- EJB has a technique that keeps business logic separate from the database. In other words, the business logic can deal with business data without caring about how data is stored in database.

- There are two main purposes of the container firstly control the EJB it contains and secondly provide the services to the EJB that are currently present in the container.
- The EJBs are typically one part of a three tier architecture as shown below.

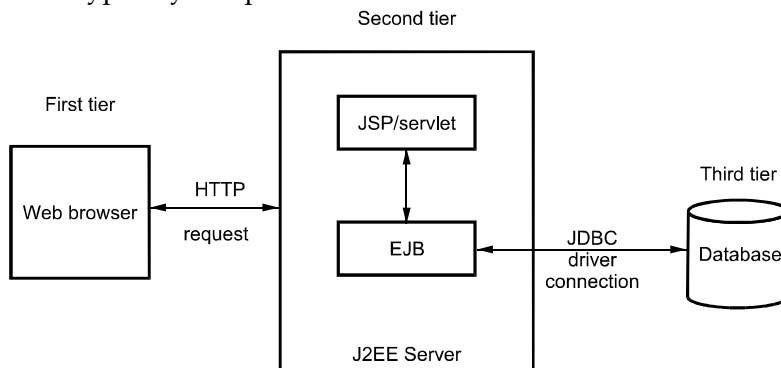


Fig. 6.17.3 3 Tier architecture

- **The following are the flows of the EJB architecture.**
 1. The client is working on a web browser.
 2. There is a database server that hosts a database, like MySQL / Oracle.
 3. The J2EE server machine is running on an application server.
 4. The client interface is provided with JSP / Servlet. The enterprise beans reside in the business tier providing to the client tier.
 5. The Application Server manages the relationships between the client and database machines.

6.17.4 EJB Technology

- Every EJB has three classes :
 1. A home interface which acts as a factory of remote objects.
 2. A remote object which is used for client interaction.
 3. A bean object which contains all the business logic.

Setting Up the Environment

- In this section we will discuss how to develop an EJB stateless session bean.
- For that purpose we need to set up the environment for execution of EJB stateless bean.
- We need following software to be installed in our machine -
 - JDK
 - NetBeans or Eclipse
 - GlassFish

- I have used NetBeans. The NetBeans is basically an Integrated Development Environment (IDE). It is useful for developing various web applications using Java, PHP, C++, EJB, mobile applications and so on. It is an open source technology. That means you can get it downloaded freely from the net. Following steps can be followed for setting up the environment before creating our bean application.

Step 1 : The NetBeans can be downloaded from <https://netbeans.org/downloads/>

Now as I want **GlassFish server** for supporting my EJB app, I will choose **Java EE bundle** for downloading. Here is the screenshot of download page of NetBeans -

NetBeans IDE 8.2 Download 8.1 | 8.2 | Development | Archive

Email address (optional):

Subscribe to newsletters: ☒ Monthly ☐ Weekly

☒ NetBeans can contact me at this address

IDE Language: English Platform: Windows

Note: Greyed out technologies are not supported for this platform.

NetBeans IDE Download Bundles

Supported technologies *	Java SE	Java EE	HTML5/JavaScript	PHP	C/C++	All
NetBeans Platform SDK	•	•				•
Java SE	•	•				•
Java FX	•	•				•
Java EE		•				•
Java ME						•
HTML5/JavaScript		•	•	•		•
PHP			•	•		•
C/C++					•	•
Groovy						•
Java Card™ 3 Connected						•
Bundled servers						
GlassFish Server Open Source Edition 4.1.1		•				•
Apache Tomcat 8.0.27		•				•

Download buttons: Download Download Download x86 Download x86 Download x86 Download x64 Download x64 Download x64 Download

Step 2 : The exe file will start downloading on your machine. After getting the exe file, click on it and start running the installer.



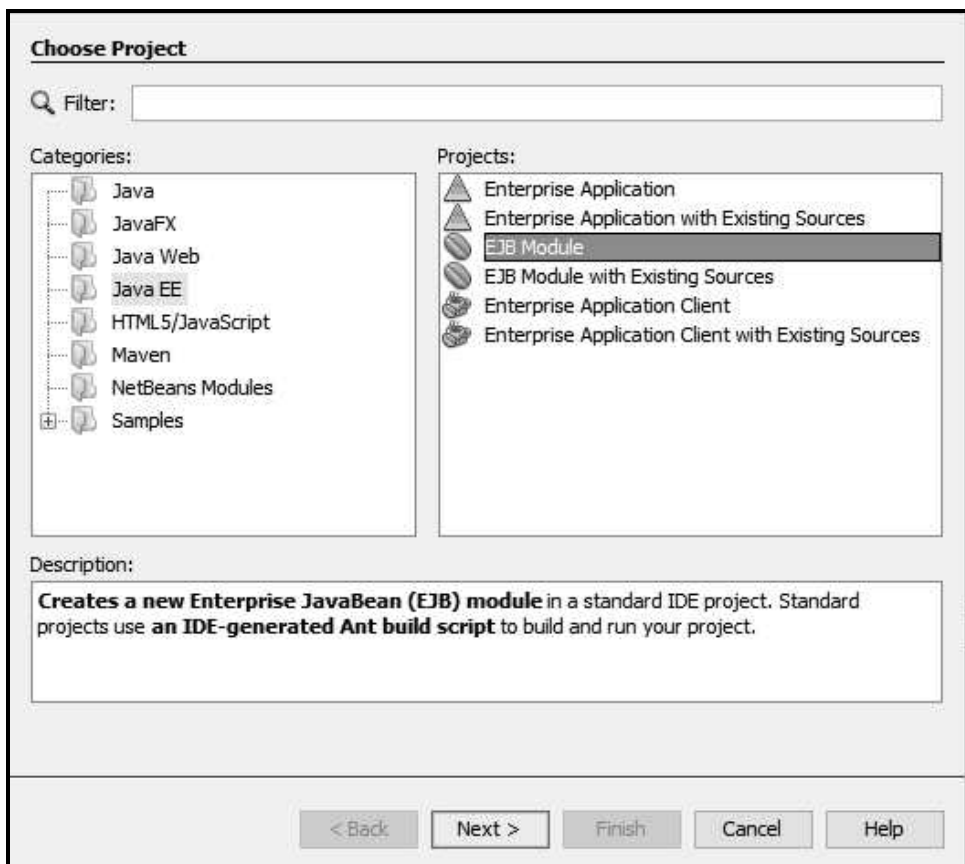
As shown in above screen-shot, Just check for GlassFish Server option. The GlassFish is an Open Source server project started by Sun-Microsystems. It supports the implementation of Enterprise Java Beans, JSF, RMI, JSP and Servlets.

Step 3 : Just go on installing the software by clicking Next button. Just uncheck for Updates if you do not want any updates for NetBeans. Finally uncheck for “Contribute to the NetBeans project by providing ... ” and click Finish button. Here completes the installation of your NetBeans software.

Creating a Simple EJB Stateless session Bean

We will follow the following steps for creation of this stateless bean -
Open NetBeans IDE

Step 1 : Select **File->New Project->Java EE-> EJB Module** and Click **Next** Button.



- Give the Suitable Name to the project. Here I am giving “MyAddApp”. Click Next Button.
- Select the server as GlassFish.
- Select the J2EE version what ever is installed on your machine and then click Finish button.

Step 2 : Create a new Java Package called myPkg (I have given this name to my package. You can give any suitable name) under the Source Package Folder.

Then create an interface called “MyInterface” under myPkg package and add following code.

MyInterface.java

```
packagemyPkg;
import javax.ejb.Remote;
@Remote
public interface MyInterface {
```



```
public int add(int a,int b);  
}
```

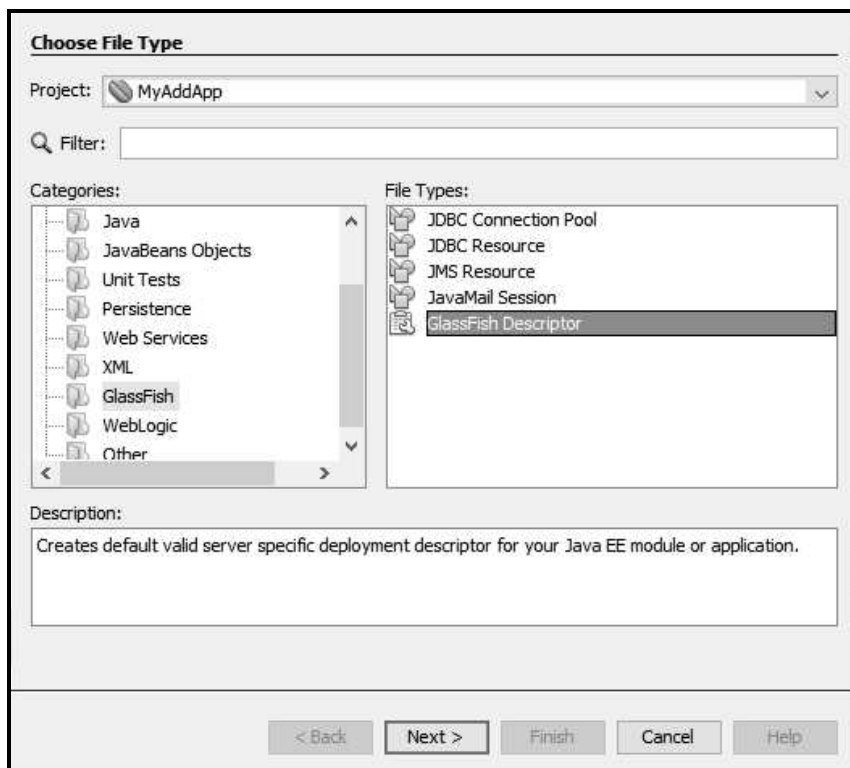
Step 3 : Right click on myPkg, and add the Java class for implementation of above interface. I have named this class as **MyClass**. This class contains the definition of the function **add** which we have declared in above interface. The code is as follows -

MyClass.java

```
package myPkg;  
import javax.ejb.Stateless;  
@Stateless  
public class MyClass implements MyInterface{  
    @Override  
    public int add(int a,int b) {  
        return a+b;  
    }  
}
```

Step 4 : We will create deployment descriptor file i.e the GlassFish Descriptor file.

Right Click on the Project "MyAddApp" and then choose **New -->Other-->GlassFish-->GlassFish Descriptor** and then click the "Next" button and click the "Finish " Button.



The **glassfish-ejb-jar.xml** file will be created under the Configuration Files Folder.

Set the JNDI Name for the **MyClass** Bean by editing the file or by clicking the TAB "EJB" and giving jndi name as "aaa". The edited xml file is as shown below -

Glassfish-ejb-jar.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE glassfish-ejb-jar PUBLIC "-//GlassFish.org//DTD GlassFish Application Server 3.1
EJB 3.1//EN" "http://glassfish.org/dtds/glassfish-ejb-jar_3_1-1.dtd">
<glassfish-ejb-jar>
  <enterprise-beans>
    <ejb>
      <ejb-name>MyClass</ejb-name>
      <jndi-name>aaa</jndi-name>
    </ejb>
  </enterprise-beans>
</glassfish-ejb-jar>
```

Step 5 : Right click on Project node(MyAddApp) and select **Clean and Build**.

After clean and Build again right click and click on project node and select **deploy**

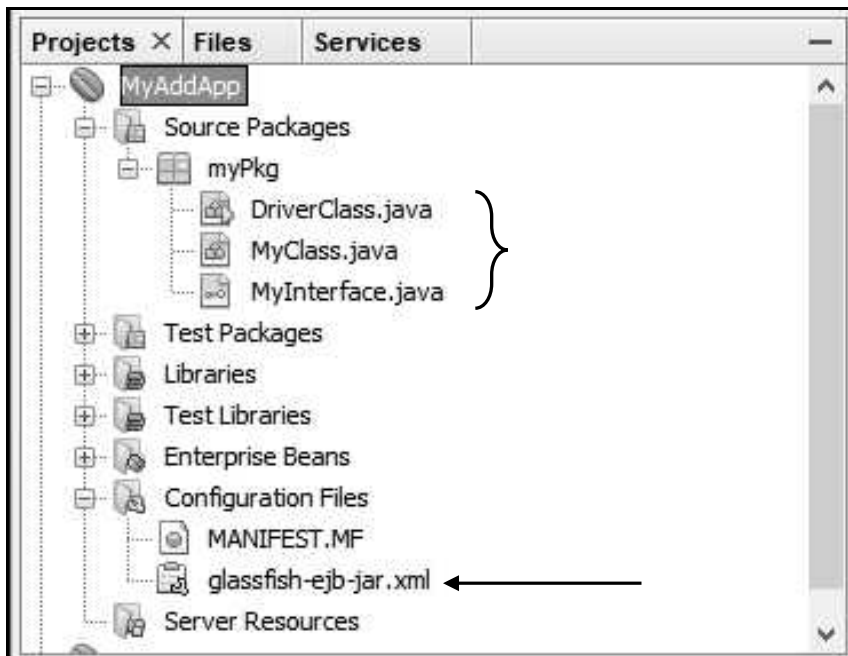
For **both** these actions "**BUILD SUCCESSFUL**" message will be displayed in the console output. Thus the JNDI name "aaa" is successfully deployed with our EJB bean **MyClass**.

Step 6 : Now it's the time to create the driver class which will call the method **add** which we have defined in our bean program **MyClass.java**(Refer step 3). For that purpose, again right click on the **myPkg** and create a Java Class. I have named it as **DriverClass** . This program will contain the JNDI name and call to the method **add** inside the **public void main** function. The code is as follows -

DriverClass.java

```
package myPkg;
import java.util.Properties;
import javax.naming.InitialContext;
public class DriverClass {
    public static void main(String arg[])throws Exception
    {
        Properties p=new Properties();
        InitialContext c=new InitialContext();
        Object obj=c.lookup("aaa");
        MyInterface inter=(MyInterface)obj;
        System.out.println("The additon of two numbers is = "+inter.add(10,20));
    }
}
```

To let you know, the locations of these files I am giving the screen-shot of Project explorer window for **MyAddApp** project.



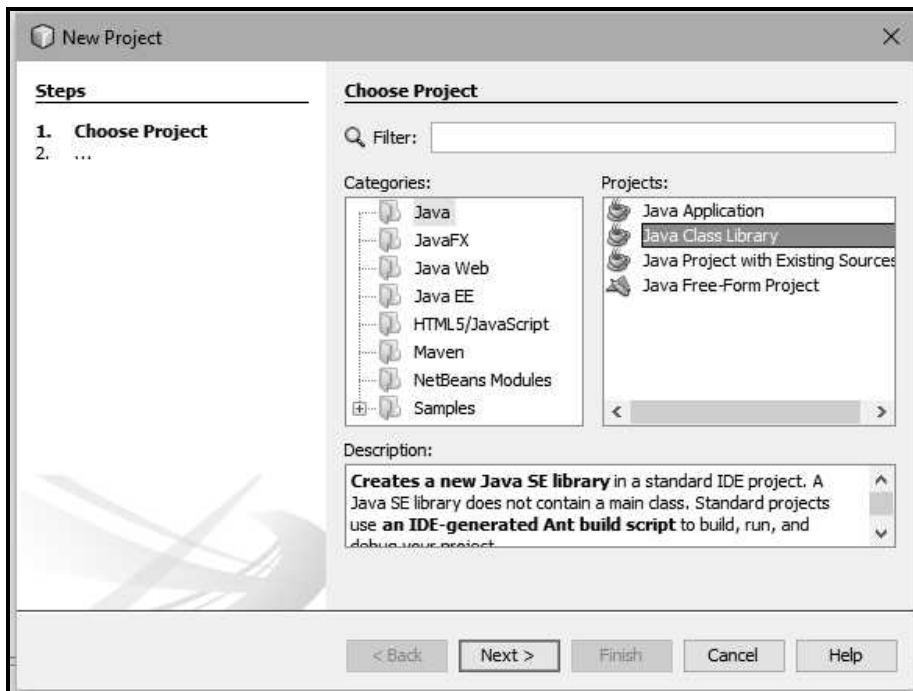
Step 7 : Right click on **DriverClass.java** click on **Run File** option and you will get the output on console window. Here is the output screenshot.



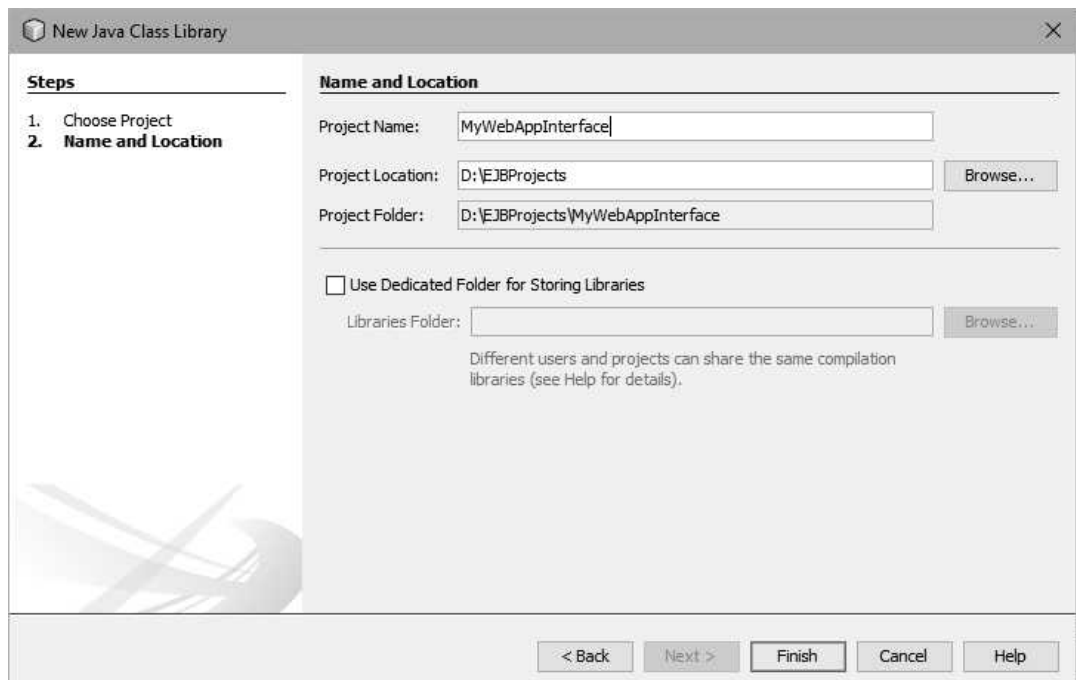
Creation of Simple Web Application using Stateful Session Bean

In this section we will learn how to create application using stateful session bean.

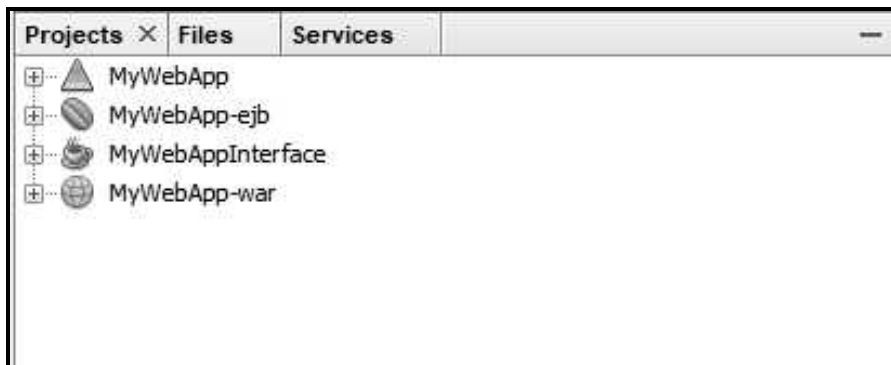
Step 1 : Click File-> New Project -> Java-> Java Class Library.



Click Next button. Add name to the Project. I have given the name as **MyWebAppInterface** and click **Finish** button.



Step 2 : Now click **File->New Project->Select ->Java EE->Enterprise Application**. Then click Next button. Give suitable name to your project. I have given the name **MyWebApp**. Then click Next button. Then just click **Finish** Button. Your Project explorer Window will look like this -

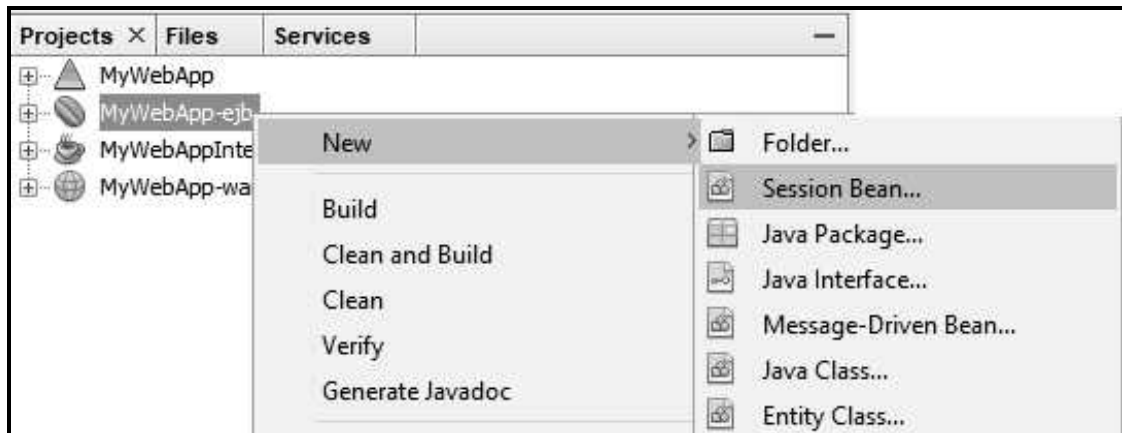


Step 3 : Now click on **MyWebApp-War** node, then click on **Web-Pages** folder, locate the **index.html** file. Double click this file and edit it as follows -

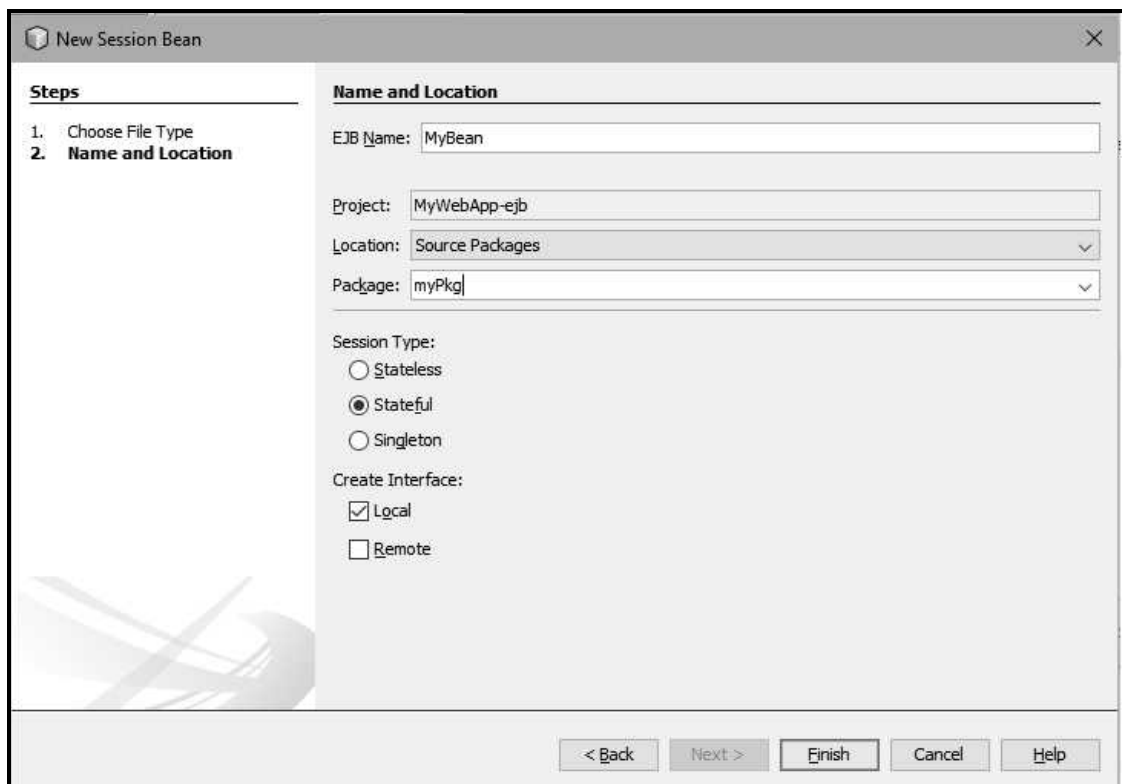
index.html

```
<html>
<head>
  <title>Web Application For Bank Transactions</title>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
</head>
<body>
  <form action="Main">
    Enter Amount: <input type="text" name="amt_txt">
    <br/> <br/>
    Select Operation:
    <br/> <br/>
    <select name="operation">
      <option value="deposit">Deposit</option>
      <option value="withdraw">Withdraw</option>
      <option value="balance">Balance</option>
    </select>
    <br/> <br/>
    <input type="Submit" value="Submit">
  </form>
</body>
</html>
```

Step 4 : Now right-click on **MyWebApp-ejb** node, click New->Session bean



Now give suitable name to your bean. Create this bean inside some package. I have created a bean named **MyBean** inside the package named **mypkg**.



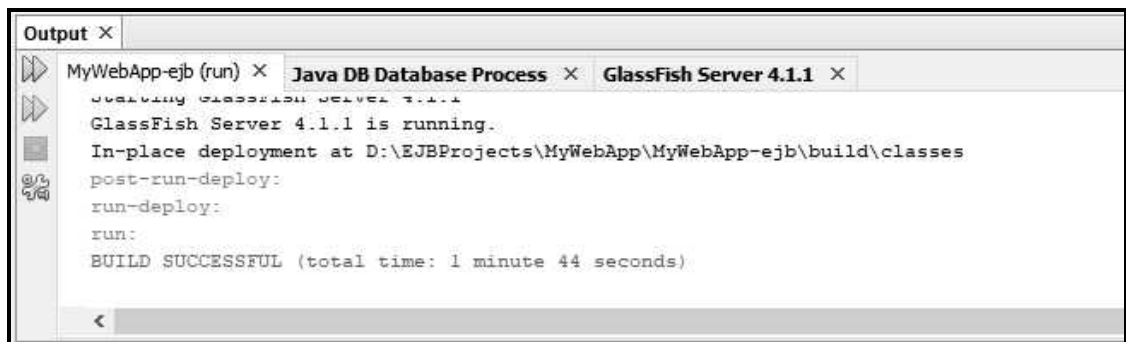
Since this is a **stateful EJB application**, Select the option **Stateful**. Here I have created a Local interface. For that purpose, **check** for **Local** interface option. Finally click **Finish** button. Edit the bean file as follows.

While inserting the code we can right click inside the class **MyBean** and add business methods as -

MyBean.java

```
package myPkg;
import javax.ejb.Stateful;
@Stateful
public class MyBean implements MyBeanLocal {
    int balance=0;
    @Override
    public void deposit(int amount) {
        balance=balance+amount;
    }
    @Override
    public boolean withdraw(int amount) {
        if(amount <= balance)
        {
            balance=balance-amount;
            return true;
        }
        return false;
    }
    @Override
    public int getBalance() {
        return balance;
    }
}
```

Step 5 : Right click **MyWebApp-ejb** node and click **Run**. The GlassFish Server will start running and you will get **BUILD SUCCESSFUL** message in console window at the bottom of your NetBean IDE.



Step 6 : Now , inside the MyWebApp-War node there is a **Source Packages** , right click it and create Servlet. I have named this as **Main**. Click the next button and Check “Add information to deployment...” as shown -

New Servlet

Steps

1. Choose File Type
2. Name and Location
3. **Configure Servlet Deployment**

Configure Servlet Deployment

Register the Servlet with the application by giving the Servlet an internal name (Servlet Name). Then specify patterns that identify the URLs that invoke the Servlet. Separate multiple patterns with commas.

☒ Add information to deployment descriptor (web.xml)

Class Name:

Servlet Name:

URL Pattern(s):

Initialization Parameters:

Name	Value
------	-------

New Edit... Delete

< Back Next > Finish Cancel Help

- Click **Finish** Button.

- The servlet code window will appear with default code. Locate the method void ProcessRequest(...) inside this default code. Just above this method, right click and select Insert Code and select Call Enterprise Bean and then select MyWebApp-ejb node and select MyBean inside it. Click OK button. Due to this some built in code for Local Interface reference will appear inside your servlet code. This code will be for the lookup method.
- Inside this code you have to locate the instance for this Local lookup. This instance is generally located at the beginning of servlet definition as follows.

Generate
Constructor...
Logger...
toString...
Override Method...
Add Property...
Call Enterprise Bean...
Use Database...
Send JMS Message...
Send E-mail...
Call Web Service Operation...
Generate REST Client...


```
import javax.servlet.http.HttpServlet;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;
import myPkg.MyBeanLocal;

/**
 *
 * @author Puntambekar
 */
public class Main extends HttpServlet {

    MyBeanLocal myBean = lookupMyBeanLocal();
```

Now using the instance **myBean** we can invoke the methods defined in our Java Bean. Note that this Java bean we have created in step 4.

The edited code for servlet is as follows -

Main.java

```
import java.io.IOException;
import java.io.PrintWriter;
import java.util.logging.Level;
import java.util.logging.Logger;
import javax.naming.Context;
import javax.naming.InitialContext;
import javax.naming.NamingException;
import javax.servlet.ServletException;
import javax.servlet.http.HttpServlet;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;
import myPkg.MyBeanLocal;

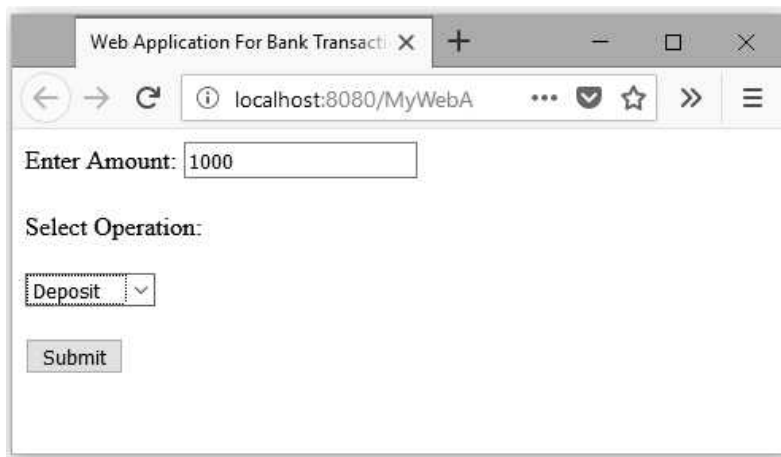
public class Main extends HttpServlet {

    MyBeanLocal myBean = lookupMyBeanLocal();

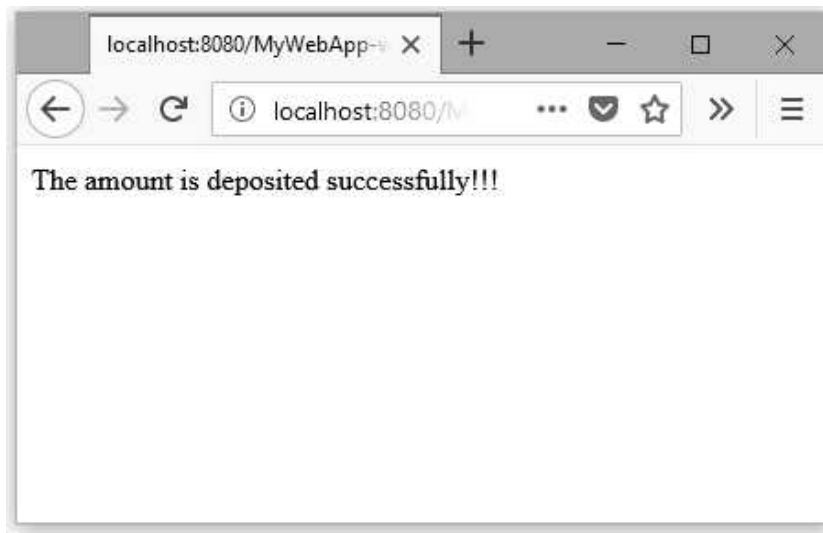
    protected void processRequest(HttpServletRequest request, HttpServletResponse response)
        throws ServletException, IOException {
        response.setContentType("text/html;charset=UTF-8");
        try (PrintWriter out = response.getWriter()) {
            String choice=request.getParameter("operation");
            int amount=Integer.parseInt(request.getParameter("amt_txt"));
            if(choice.equals("deposit"))
            {
                myBean.deposit(amount);
                out.println("The amount is deposited successfully!!!");
            }
            if(choice.equals("withdraw"))
            {
```

```
        if(myBean.withdraw(amount))
        {
            out.println("The amount is withdrawn successfully!!!");
        }
        Else
            out.println("Sorry, Balance insufficient!!!");
    }
    if(choice.equals("balance"))
    {
        out.println("The total balance is = "+myBean.getBalance());
    }
    }
}
...
....
private MyBeanLocal lookupMyBeanLocal() {
    try {
        Context c = new InitialContext();
        return (MyBeanLocal) c.lookup("java:global/MyWebApp/MyWebApp-
ejb/MyBean!myPkg.MyBeanLocal");
    } catch (NamingException ne) {
        Logger.getLogger(getClass().getName()).log(Level.SEVERE, "exception caught", ne);
        throw new RuntimeException(ne);
    }
}
}
```

Step 7 : Now right click the **MyWebApp-ejb** and deploy it. Again click on the node **MyWebApp** and Run. The browser window will execute the application as follows -



Select the operations as perform the transactions.



6.17.5 EJB Container and its Services

- The EJB container provides following services to EJB component -

Persistence

- The EJB container provides the persistence service. That means, EJB can save its state in web container. The persistence services include simple connection pooling, reading some data from databases to generate dynamic pages or update some simple information. Writing data access code is error prone and time consuming. Hence the enterprise beans are created by application development tools in such a way that the data access logic will be encapsulated within the components. Due to this service the real-world project can be benefited in terms of time, budget and personal constraints.

Declarative Transactions

- Transaction management is complex if multiple data access components or multiple data sources are involved. The EJB provides the transaction management service in such a way that the complex transaction can be managed without any coding.
- The Java API - **Java Transaction API (JTA)** provides standard interface between transaction manager, application server and component. There is another Java API **Java Transaction Service (JTS)** which is directly used by the application programmer.

Declarative Security

- It is most essential to have security for data and business logic functionality. Usually application developers provide security using servlets or JSP pages. But this is complex and error prone task. The EJB uses the simplified security model due to which EJB resources can be protected from unauthorized access. The EJB developer rely on security provided at the web level.

Error Handling

- The EJB offers clear and consistent error handling framework. The EJB specification defines how errors affect transactions, client results, server logging and recovery of components.

Component Framework for Business Logic

- The software that requires a complex business logic require large number of enterprise resources. EJB are server side components that can be used simultaneously by different clients. Using the EJB's common framework for the business logic the building of complex application is possible.

Scalability

- Using EJB specification rules it is possible for the application server to manage large number of simultaneous clients. EJB specification is useful to satisfy the multiple demands for business logic components and data access components.

Portability

- EJB's are written to an industry standard API. Hence they can be run on any new J2EE compliant application server. There is no need to modify the EJB components for running them on application servers.

Manageability

- EJB container provides the facility to manage access, add or remove users from appropriate role. Using EJB it is possible to manage secure access to every user interface that provides the functionality of accessing the users.

6.17.6 EJB Interfaces

- In EJB, the client can access the session bean using the business interface. Depending upon the location of client, there are two types of interfaces used -

- 1) **Remote Interface** : The remote interface is used by client that may be running on different machine than the bean. Using Remote Method Interface (RMI) the access to the bean can be made.
- 2) **Local Interface** : The local interface can be used by the client which is running on the same machine where the bean is residing.

Various aspects while deciding between local and remote interface are represented in the following table -

Aspect	Use of Remote Interface	Use of Local Interface
Type of client	If the clients are application clients, web components or EJBs executing on different machine.	If the clients are either web components or EJBs executing on the same machine.
Protocol used	If the RMI used for communication.	If the local function calls are used for communication.
Component distribution	If the component in the application are distributed, that means present on different machines.	If the components are located on the same machine where bean is present.
Loose or tight coupling	If the client and bean are coupled loosely.	If there is a requirement of tight coupling between EJB components.
Performance	Slow performance due to RMI calls.	Better performance due to local calls.
Location transparency	If the client need not know the location of EJB and access it transparently.	If the client wants to know that the current machine is the location of EJB.
Communication	If multiple objects are passed by value so that client and bean can operate on different copies of the object.	If objects are passed by reference and the client or bean can change the value of the same object.
Serialization	If the serialized objects are passed.	If the objects are not serialized.

6.17.7 EJB Clients

- EJB is the server-side and platform-independent Java Application Programming Interface (API) for Java Platform, Enterprise Edition (Java EE). EJB is used to simplify the development of large distributed applications. EJB client is a code which will use the EJB functionality. Clients cannot create objects of EJB class component. Clients can access any EJB only through the EJB container. Some of the EJB clients are as follows -
 - 1) Java Applications
 - 2) Other EJBs on same or another application server.
 - 3) Java Applets
 - 4) A web service can access EJB
 - 5) Servlets, JSP- some server side technologies.

Review Questions

1. *Decide and explain various aspects while deciding between local and remote interface can be considered.* **SPPU : May-18, Marks 8**
2. *What is enterprise Java Bean ? Draw and explain main components of EJB architecture.* **SPPU : May-18, Marks 8**
3. *Draw and explain the role of EJB container in enterprise applications.* **SPPU : May-18, Marks 6**
4. *Draw and explain scenario of client accessing remote EJB. List some of the EJB clients.* **SPPU : Dec.-18, May-19, Marks 8**
5. *Draw and explain each tier of three tiers architecture using EJB.* **SPPU : May-19, Marks 8**

6.18 Multiple Choice Questions

Q.1 Ruby is a ____.

- ☐ a) scripting language
- ☐ b) server side programming language
- ☐ c) object oriented programming language
- ☐ d) procedural programming language

Q.2 ____ method in ruby removes the last character.

- ☐ a) Pop
- ☐ b) Remove
- ☐ c) Del
- ☐ d) Chop

- Q.3** ____ method in ruby is used to insert the element from the end.
- ☐ a Insert ☐ b Push
☐ c Put ☐ d None of these
- Q.4** What is the extension used for saving the ruby file ?
- ☐ a .ruby ☐ b .rb
☐ c .ry ☐ d None of these
- Q.5** Which of the following is valid conditional statement in ruby ?
- ☐ a elseif ☐ b elsif
☐ c else if ☐ d elseiff
- Q.6** What will any variable evaluate to if it is of Boolean data type ?
- ☐ a True ☐ b NULL
☐ c False ☐ d Either True or False
- Q.7** Which services are provided to EJB components by the EJB container ?
- ☐ a Transaction support ☐ b Persistence support
☐ c Naming support ☐ d All mentioned above
- Q.8** Session beans don't have ____.
- ☐ a ejbCreate() method ☐ b ejbStore() method
☐ c ejbRemove() method ☐ d none of these
- Q.9** An entity bean's local interface MUST extend the _____ interface.
- ☐ a javax.ejb.EJBLocalObject ☐ b javax.ejb.EJBObject
☐ c javax.ejb.RemoteObject ☐ d none of the above
- Q.10** What executes EJB components ?
- ☐ a A web server ☐ b An application server
☐ c An EJB container ☐ d A database server
- Q.11** What represents a persistent global data from the database ?
- ☐ a Entity bean ☐ b Session bean
☐ c Both A & B ☐ d None of the above

Answer Keys for Multiple Choice Questions :

Q.1	c	Q.2	d	Q.3	b	Q.4	b
Q.5	b	Q.6	d	Q.7	d	Q.8	a
Q.9	a	Q.10	c	Q.11	a		



Notes

[illegible]

Web Technology Laboratory

Contents

- Experiment 1 :* Design the XML document to store the information of the employees of any business organization and demonstrate the use of :
a) DTD b) XML Schema
And display the content in (e.g., tabular format) by using CSS/XSL.....L - 2
- Experiment 2 :* Implement an application in Java Script using following :
a) Design UI of application using HTML, CSS etc.
b) Include Java script validation
c) Use of prompt and alert window using Java Script
e.g., Design and implement a simple calculator using Java Script for operations like addition, multiplication, subtraction, division, square of number etc.
a) Design calculator interface like text field for input and output, buttons for numbers and operators etc.
b) Validate input values
c) Prompt/alerts for invalid values etcL - 4
- Experiment 3 :* Implement the sample program demonstrating the use of Servlet. e.g., create a database table ebookshop (book_id, book_title, book_author, book_price, quantity) using database like Oracle/MySQL etc. and display (use SQL select query) the table content using servlet.....L - 9
- Experiment 4 :* Implement the program demonstrating the use of JSP. e.g., Create a database table students_info (stud_id, stud_name, class, division, city) using database like Oracle/MySQL etc. and display (use SQL select query) the table content using JSP.....L - 9
- Experiment 5 :* Build a dynamic web application using PHP and MySQL.
a. Create database tables in MySQL and create connection with PHP.
b. Create the add, update, delete and retrieve functions in the PHP web app interacting with MySQL database..L - 13
- Experiment 6 :* Design a login page with entries for name, mobile number email id and login button. Use struts and performs following validations
a. Validation for correct names
b. Validation for mobile numbers
c. Validation for email id
d. Validation if no entered any value
e. Re-display for wrongly entered values with message
f. Congratulations and welcome page upon successful entries.....L - 13
- Experiment 7 :* Design an application using Angular JS. e.g., Design registration (first name, last name, username, password) and login page using Angular JS.....L - 21
- Experiment 8 :* Design and implement a business interface with necessary business logic for any web application using EJB. e.g., Design and implement the web application logic for deposit and withdraw amount transactions using EJB
.....L - 26

Experiment 1 : Design the XML document to store the information of the employees of any business organization and demonstrate the use of :

- a) DTD
- b) XML Schema

And display the content in (e.g., tabular format) by using CSS/XSL.

Solution : a) Employee DTD

Employee.dtd

```
<!DOCTYPE Person[  
<!ELEMENT Person (Employee+)>  
<!ELEMENT Employee (EmpId,EmpName,Department)>  
<!ELEMENT EmpId (#PCDATA)>  
<!ELEMENT EmpName (#PCDATA)>  
<!ELEMENT Department (#PCDATA)>  

```

b) Step 1 : The XML document containing employee information is as follows -

Employee.xml

```
<?xml version="1.0" encoding="UTF-8"?>  
<?xml-stylesheet type="text/xsl" href="EmployeeSchema.xsl" ?>  
<Person>  
<Employee>  
  <EmpId>111</EmpId>  
  <EmpName>Archana</EmpName>  
  <Department>Accounts</Department>  
  
</Employee>  
<Employee>  
  <EmpId>222</EmpId>  
  <EmpName>Rashmi</EmpName>  
  <Department>Purchase</Department>  
  
</Employee>  
<Employee>  
  <EmpId>333</EmpId>  
  <EmpName>Sheetal</EmpName>  
  <Department>Sales</Department>  
  
</Employee>  
<Employee>  
  <EmpId>444</EmpId>  
  <EmpName>Aniket</EmpName>  
  <Department>Marketing</Department>
```

```
</Employee>  
</Person>
```

Step 2 : Display of information using xsl is done using following .xsl document.

Employeeschema.xsl

```
<?xml version="1.0" encoding="ISO-8859-1"?>  
  
<xsl:stylesheet version="1.0"  
xmlns:xsl="http://www.w3.org/1999/XSL/Transform">  
  
<xsl:template match="/">  
  <html>  
    <body>  
      <center>  
        <h2>Employee Information</h2>  
        <table border="1">  
          <tr bgcolor="gray">  
            <th>EmpId</th>  
            <th>EmpName</th>  
            <th>Department</th>  
          </tr>  
          <xsl:for-each select="Person/Employee">  
            <tr>  
              <td bgcolor="green"><xsl:value-of select="EmpId"/></td>  
              <td bgcolor="red"><b><xsl:value-of  
select="translate(EmpName,'abcdefghijklmnopqrstuvwxyz','ABCDEFGHIJKLMNOPQRSTUVWXYZ')"/>  
</b></td>  
              <td bgcolor="cyan"><xsl:value-of select="Department"/></td>  
            </tr>  
          </xsl:for-each>  
        </table>  
      </center>  
    </body>  
  </html>  
</xsl:template>  
</xsl:stylesheet>
```

Experiment 2 : Implement an application in Java Script using following :

- a) Design UI of application using HTML, CSS etc.
- b) Include Java script validation
- c) Use of prompt and alert window using Java Script

e.g., Design and implement a simple calculator using Java Script for operations like addition, multiplication, subtraction, division, square of number etc.

- a) Design calculator interface like text field for input and output, buttons for numbers and operators etc.
- b) Validate input values
- c) Prompt/alerts for invalid values etc.

Solution :

calci.html

```
<!DOCTYPE html>
<html>
<head>
<title> JavaScript and CSS Calculator </title>
<style>
.form_style
{
    width: 300px;
    height: 580px;
    margin: auto;
    border: 5px solid blue;
    border-radius: 5px;
    padding: 20px;
}
input
{
    width: 20px;
    background-color: grey;
    color: white;
    border: 3px solid black;
    border-radius: 5px;
    padding: 28px;
    margin: 5px;
    font-weight: bold;
    font-size: 15px;
}
#calculate
{
    width: 250px;
    background-color: khaki;
    color: black;
```

```
border: 5px solid black;
border-radius: 3px;
padding: 20px;
margin: auto;
}
#equal_to
{
width: 210px;
border: 3px solid gray;
border-radius: 3px;
padding: 26px;
background-color: skyblue;
}
</style>
</head>
<body>
<div class= "form_style">
<form name = "myform">
<input id = "calculate" type ="text" name = "answer"> <br> <br>
<input type = "button" value = "7" onclick = "display('7')">
<input type = "button" value = "8" onclick = "display('8')">
<input type = "button" value = "9" onclick = "display('9')">
<input type = "button" value = "+" onclick = "display('+') ">
<br> <br>

<input type = "button" value = "4" onclick = "display('4')">
<input type = "button" value = "5" onclick = "display('5')">
<input type = "button" value = "6" onclick = "display('6')">
<input type = "button" value = "-" onclick = "display('-')">
<br> <br>

<input type = "button" value = "1" onclick = "display('1') ">
<input type = "button" value = "2" onclick = "display('2') ">
<input type = "button" value = "3" onclick = "display('3') ">
<input type = "button" value = "*" onclick = "display('*')">
<br> <br>

<input type = "button" value = "0" onclick = "display('0')">
<input type = "button" value = "." onclick = "display('.')">
<input type = "button" value = "C" onclick = "clear_all() ">
<input type = "button" value = "/" onclick = "display('/')">
<br>

<input type = "button" value = "SQ" onclick = "square_val() ">
```

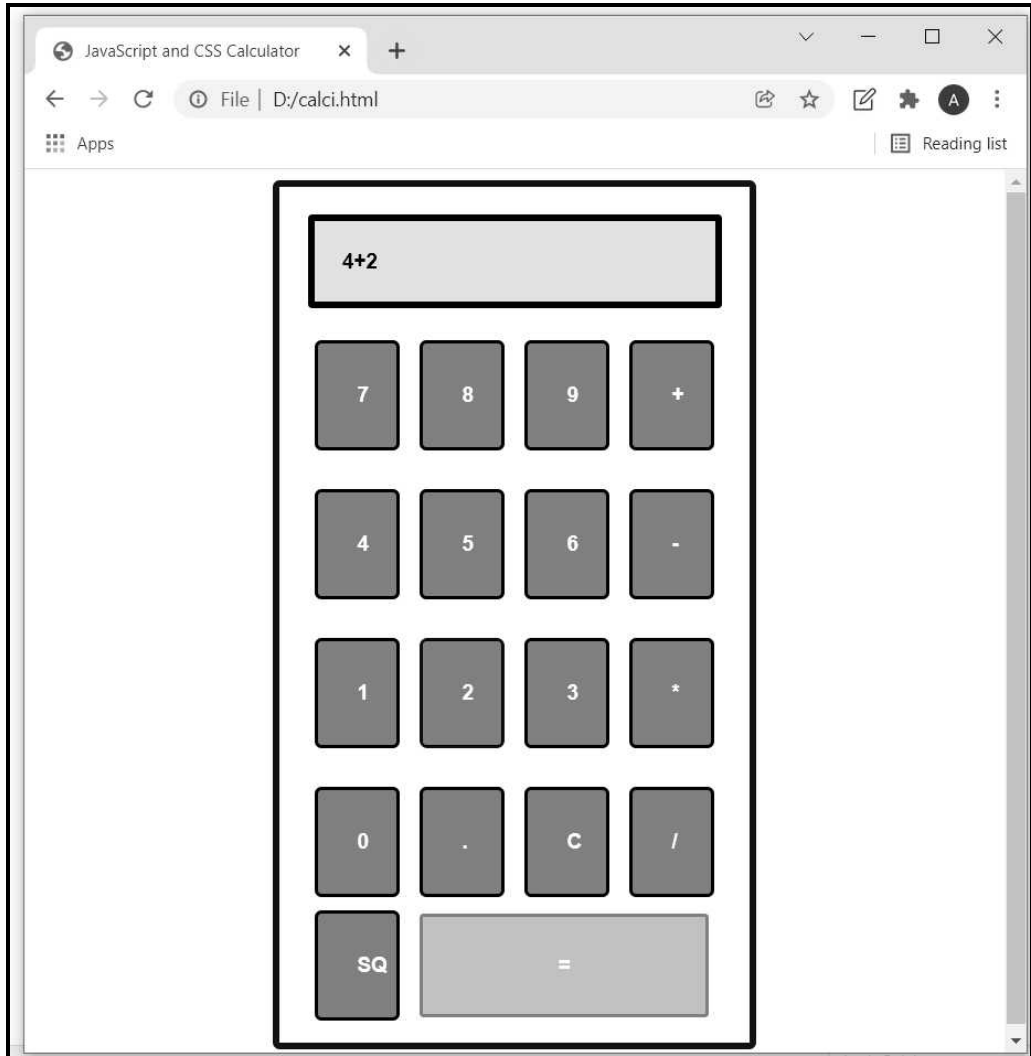
```
<input type = "button" value = "=" onclick = "result()" id = "equal_to" >
</form>
</div>
<script>
    function display(val)
    {
        myform.answer.value +=val
    }

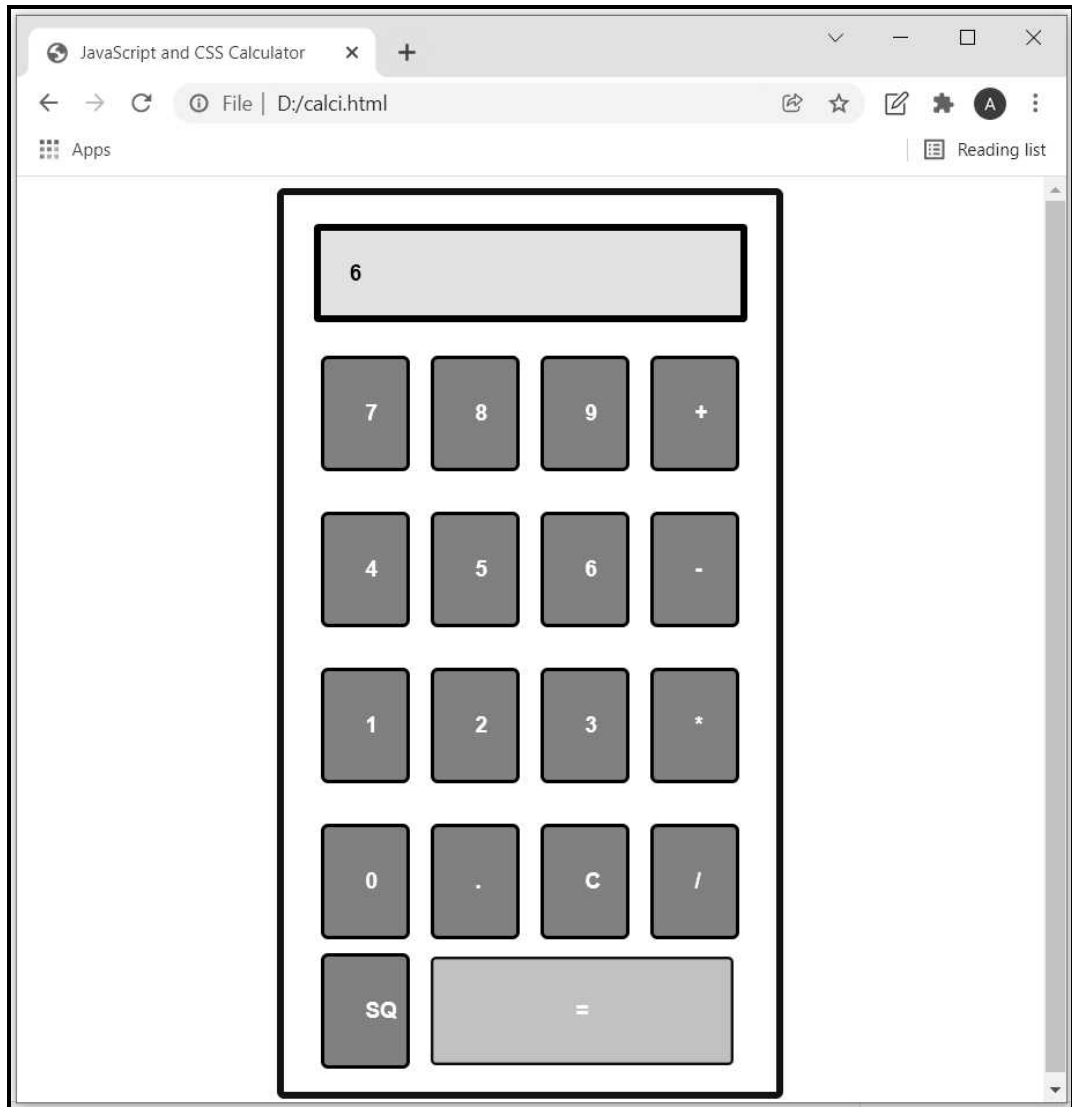
    function clear_all()
    {
        alert("You have clicked clear all button")
        myform.answer.value = ''
    }

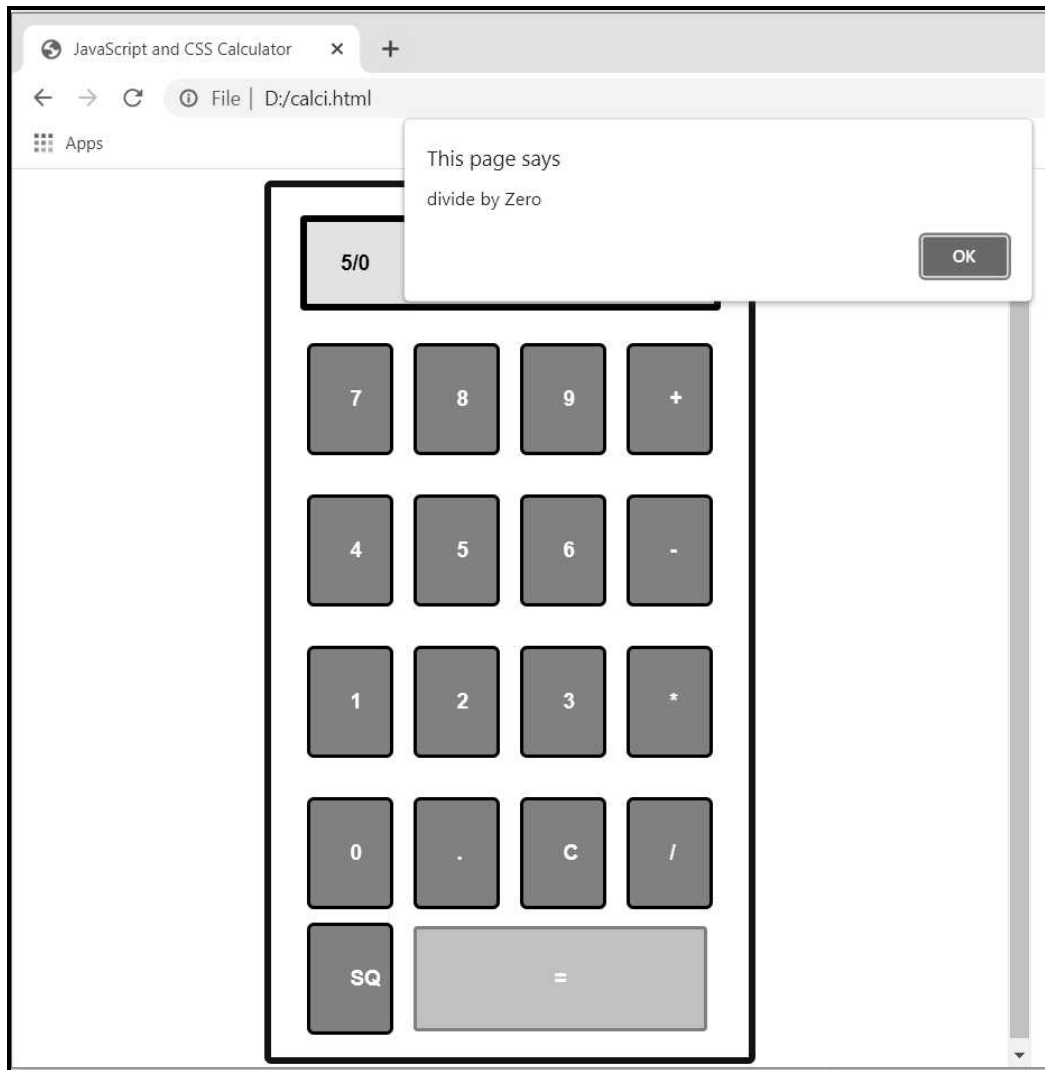
    function square_val()
    {
        num=myform.answer.value
        myform.answer.value = num*num
    }

    function result()
    {
        ans = eval(myform.answer.value)
        if(ans==Infinity)
        {
            alert("divide by Zero")
            myform.answer.value=''
        }
        if(ans==undefined)
        {
            alert("Enter some input")
            myform.answer.value=''
        }
        else
            myform.answer.value = ans
    }
</script>
</body>
</html>
```

Output







Experiment 3 : Implement the sample program demonstrating the use of Servlet. e.g., Create a database table ebookshop (book_id, book_title, book_author, book_price, quantity) using database like Oracle/MySQL etc. and display (use SQL select query) the table content using servlet.

Solution : Refer similar example 3.14.2.

Experiment 4 : Implement the program demonstrating the use of JSP. e.g., Create a database table students_info (stud_id, stud_name, class, division, city) using database like Oracle/MySQL etc. and display (use SQL select query) the table content using JSP.

Solution :

Step 1 : Create a database named registration_DB using following command on the MYSQL prompt

```
mysql>CREATE DATABASE registration_DB;
```

Then create a table under the database registration_DB as follows -

```
mysql>USE registration_DB;  
mysql>CREATE TABLE registration_table(id INT(4) NOT NULL AUTO_INCREMENT, name  
VARCHAR(50) NOT NULL,  
email VARCHAR(50) NOT NULL,  
phone VARCHAR(10) NOT NULL,  
PRIMARY KEY(id));
```

The created table fields can be seen using following command.

```
mysql>DESCRIBE registration_table;
```

Step 2 : The JSP page containing the input entry form is as given below. On this JSP page itself we are retrieving the values from the database by connection to it.

Registration_Form.jsp

```
<%@ page language="java" import="java.sql.*" %>  
<%@ page import="java.io.*" %>  
    <%  
        Connection conn=null;  
        ResultSet rs=null;  
        Statement stmt=null;  
        Class.forName("com.mysql.jdbc.Driver").newInstance();  
        conn=DriverManager.getConnection("jdbc:mysql://localhost:3306/registration_DB",  
            "root","system");  
        out.write("Connected to mysql!!!");  
        stmt=conn.createStatement();  
        if(request.getParameter("action") != null)  
        {  
            String Studname=request.getParameter("Studname");  
            String Studemail=request.getParameter("Studemail");  
            String Studphone=request.getParameter("Studphone");  
            stmt.executeUpdate("insert into registration_table(name,email,phone) values(' "+Studname+"  
            ',' "+Studemail+" ',' "+Studphone+" ' )");  
            rs=stmt.executeQuery("select * from registration_table");  
        }  
    %>  
    <html>  
    <body>  
    <center>  
        <h2>Student List</h2>
```

```
<table border="1" cellspacing="0" cellpadding="0">
<tr>
  <td><b>No</b></td>
  <td><b>User Name</b></td>
  <td><b>E-Mail</b></td>
  <td><b>Phone</b></td>
</tr>
<%
int num=1;
while(rs.next()){
%>
<tr>
  <td><%=num%></td>
  <td><%=rs.getString("name")%></td>
  <td><%=rs.getString("email")%></td>
  <td><%=rs.getString("phone")%></td>
</tr>
<%
num++;
}
rs.close();
stmt.close();
conn.close();
%>
</table>
</center>
</body>
</html>
<%>else{%>
<html>
<head>
<title>Student Registration Demo</title>
<script language="javascript">
  function validation(Form_obj)
  {
    if(Form_obj.Studname.value.length==0){
      alert("Please,fill up the remaining information!!");           Form_obj.Studname.focus();
      return false;
    }
    if(Form_obj.Studemail.value.length==0){
      alert("Please,fill up the remaining information!!");
      Form_obj.Studaddress.focus();
      return false;
    }
  }
}
```

```
if((Form_obj.Studphone.value.length==0) ||
(Form_obj.Studphone.value.length>10))
{
alert("Invalid Phone number Entry, fill up once again!!");
Form_obj.Studphone.focus();
return false;
}

return true;
}
</script>
</head>

<body>
<center>
<form action="Registration_Form.jsp" method="post"
name="entry" onSubmit="return validation(this)">
<input type="hidden" value="list" name="action">
<table border="3" cellpadding="0" cellspacing="0">
<tr>
<td>
<table>
<tr>
<td colspan="2" align="center">
<h2>Student Entry Form</h2></td>
</tr>
<tr>
<td colspan="2">&nbsp;   </td>
</tr>
<tr>
<td>Student Name:</td>
<td><input name="Studname" type="text" size="50"></td>
</tr>
<tr>
<td>Student Password:</td>
<td><input name="Studpass" type="password" size="50"></td>
</tr>
<tr>
<td>E-Mail:</td>
<td><input name="Studemail" type="text" size="50"></td>
</tr>
<tr>
<td>Phone:</td>
<td><input name="Studphone" type="text" size="15"></td>
```

```
</tr>
<tr>
<td colspan="2" align="center">
<input type="submit" value="Submit"> </td>
</tr>
</table>
</td>
</tr>
</table>
</form>
</center>
</body>
</html>
<%}%>
```

Experiment 5 : Build a dynamic web application using PHP and MySQL.

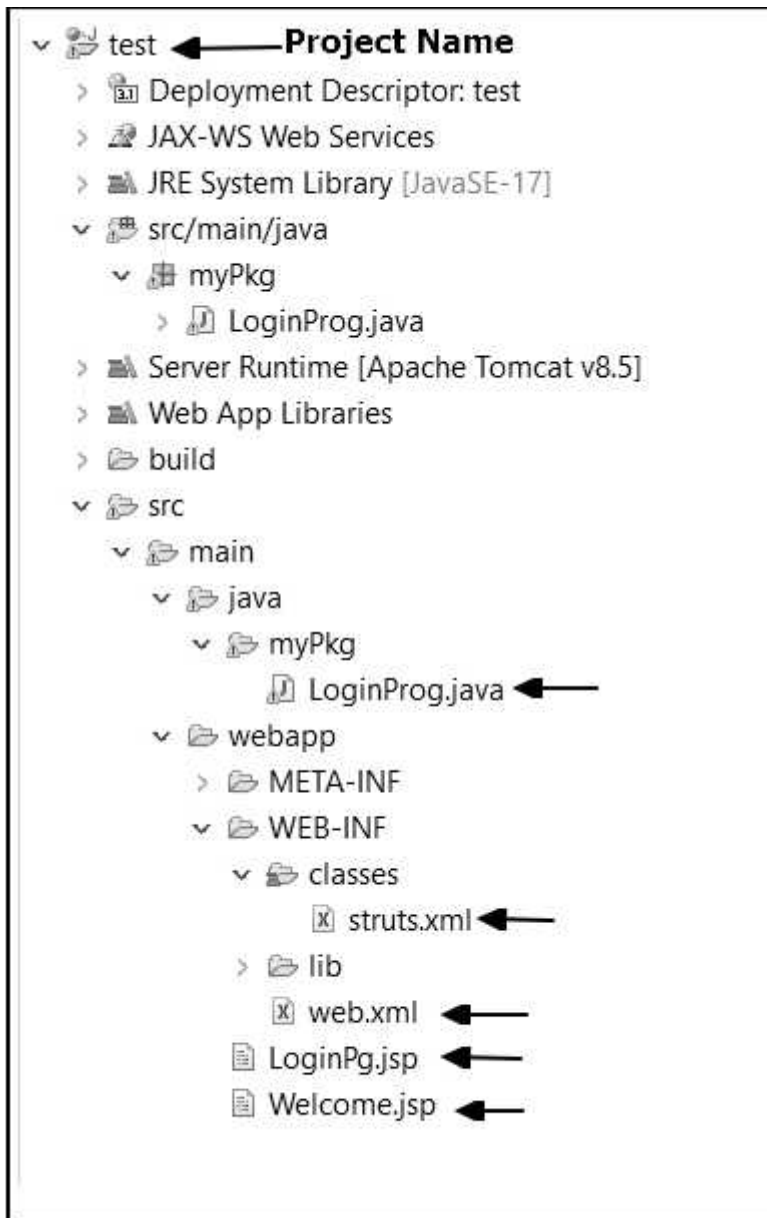
- a. Create database tables in MySQL and create connection with PHP.
- b. Create the add, update, delete and retrieve functions in the PHP web app interacting with MySQL database

Solution : Refer example 5.16.1

Experiment 6 : Design a login page with entries for name, mobile number email id and login button. Use struts and performs following validations

- a. Validation for correct names
- b. Validation for mobile numbers
- c. Validation for email id
- d. Validation if no entered any value
- e. Re-display for wrongly entered values with message
- f. Congratulations and welcome page upon successful entries

Solution : In this web application using struts2 we will create two JSP files and one Java action file. Moreover, we will create two configuration files named struts.xml and web.xml. For understanding the locations of these files refer the following screenshot.



Following are the steps to be followed to build the login validation web application.

Sep 1 : Start Eclipse. Create Dynamic Web Project. Copy the required jar files to **WEB-INF/lib** folder.

Now Create a Login Page using JSP. This file is created in **WebContent** or **webapp**. This page displays the login form with the textfields such as name, mobile number and email-id. The code for this JSP script is as follows -

LoginPg.jsp

```
<%@ page language="java" contentType="text/html; charset=ISO-8859-1"
pageEncoding="ISO-8859-1"%>
<%@taglib uri="/struts-tags" prefix="s" %>
<!DOCTYPE html PUBLIC "-//W3C//DTD HTML 4.01 Transitional//EN"
"http://www.w3.org/TR/html4/loose.dtd">
<html>
<head>
<meta http-equiv="Content-Type" content="text/html; charset=ISO-8859-1">
<title>Login Page Demo</title>
<s:head />
</head>
<body>
<s:form action="Login" validate="true">
<s:textfield name="userName" label="User Name" />
<s:textfield name="mobileNum" label="Mobile Number" />
<s:textfield name="email" label="Email ID" />

<s:submit value="Login" />
</s:form>
</body>
</html>
```

Step 2 : Create one more JSP document under **WebContent** or **webapp** folder named **Welcome.jsp**. This JSP file is for displaying welcome message for authentic user. The code is

Welcome.jsp

```
<%@ page language="java" contentType="text/html; charset=ISO-8859-1"
pageEncoding="ISO-8859-1"%>
<%@taglib uri="/struts-tags" prefix="s"%>
<!DOCTYPE html PUBLIC "-//W3C//DTD HTML 4.01 Transitional//EN"
"http://www.w3.org/TR/html4/loose.dtd">
<html>
<head>
<meta http-equiv="Content-Type" content="text/html; charset=ISO-8859-1">
<title>login Page Demo</title>
</head>
<body>
<h1>Congratulations!!! </h1>
<h2>Welcome, User <s:property value="userName"/>!!!</h2>
</body>
</html>
```

Step 3 : Now we will write the Java File for validating the user name, mobile number and email-id.

The validation for empty fields, or incorrect values is done in this program. If user enters incorrect values then the error message is displayed with the incorrect values which user has entered. If user does not enter the values and press the login button, then error message is displayed for the corresponding fields. If the user enters correct user name, correct mobile number and correct email-id then only he/she is redirected to welcome page. This validation is done inside **validate()** function

The code for this Java program is as follows -

LoginProg.java

```
package myPkg;
import com.opensymphony.xwork2.ActionSupport;

import java.util.regex.Matcher;
import java.util.regex.Pattern;

public class LoginProg extends ActionSupport{
    private String userName;
    private String mobileNum;
    private String email;
    public String execute() {
        return SUCCESS;
    }
    public String getUserName() {
        return userName;
    }
    public void setUserName(String userName) {
        this.userName = userName;
    }
    public String getMobileNum() {
        return mobileNum;
    }
    public void setMobileNum(String mobileNum) {
        this.mobileNum = mobileNum;
    }
    public String getEmail() {
        return email;
    }
    public void setEmail(String email) {
        this.email = email;
    }
}
```



```

public void validate() {
    //validating user name
    if (getUserName().length() == 0) {
        addFieldError("userName", "User Name is required");
    }
    else if (!getUserName().equals("Admin")) {
        addFieldError("userName", getText(userName)+" is not authentic user name");
    }
    //validating mobile number
    if (getMobileNum().length() == 0) {
        addFieldError("mobileNum", "Mobile Number is required");
    }
    else if (getMobileNum().length() != 10)
        addFieldError("mobileNum", getText(mobileNum)+" is invalid Mobile Number");
    else if (!getMobileNum().equals("1234567890")) {
        addFieldError("mobileNum", getText(mobileNum)+" is not authentic mobile number");
    }
    //validating email-id
    if (getEmail().length() == 0) {
        addFieldError("email", "Email-ID is required");
    }
    else if (!getEmail().equals("abc.xyz@gmail.com")) {
        addFieldError("email", getText(email)+" is not authentic email-id");
    }
    else {
        String expression="^[\\w\\-]+([\\w\\.\\w])+[\\w]+@[\\w\\-]+\\.\\.[A-Z]{2,4}$";
        CharSequence inputStr = email;
        Pattern pattern = Pattern.compile(expression,Pattern.CASE_INSENSITIVE);
        Matcher matcher = pattern.matcher(inputStr);
        if(!matcher.matches())
            addFieldError("email", "Invalid email address");
    }
}
}

```

Step 4 : Now we will write the configuration files. The struts.xml file under the **WEB-INF/classes** folder will be -

Struts.xml

```

<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE struts PUBLIC
"-//Apache Software Foundation//DTD Struts Configuration 2.0//EN"
"http://struts.apache.org/dtds/struts-2.0.dtd">

```

```
<struts>
<constant name="struts.custom.i18n.resources" value="LoginProg" />
<package name="default" extends="struts-default" namespace="/">
<action name = "Login"
class = "myPkg.LoginProg"
method = "execute">
<result name = "success">/Welcome.jsp</result>
<result name = "input">/LoginPg.jsp</result>
</action>
</package>
</struts>
```

Step 5 : The **web.xml** file present inside the WEB-INF folder is modified as -

web.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<web-app xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xmlns="http://xmlns.jcp.org/xml/ns/javaee"
xsi:schemaLocation="http://xmlns.jcp.org/xml/ns/javaee http://xmlns.jcp.org/xml/ns/javaee/web-
app_3_1.xsd"
id="WebApp_ID" version="3.1">
  <display-name>Struts2</display-name>
  <welcome-file-list>
    <welcome-file>LoginPg.jsp</welcome-file>
    <welcome-file>Welcome.jsp</welcome-file>
  </welcome-file-list>
  <filter>
    <filter-name>struts2</filter-name>
    <filter-class>
      org.apache.struts2.dispatcher.FilterDispatcher
    </filter-class>
  </filter>

  <filter-mapping>

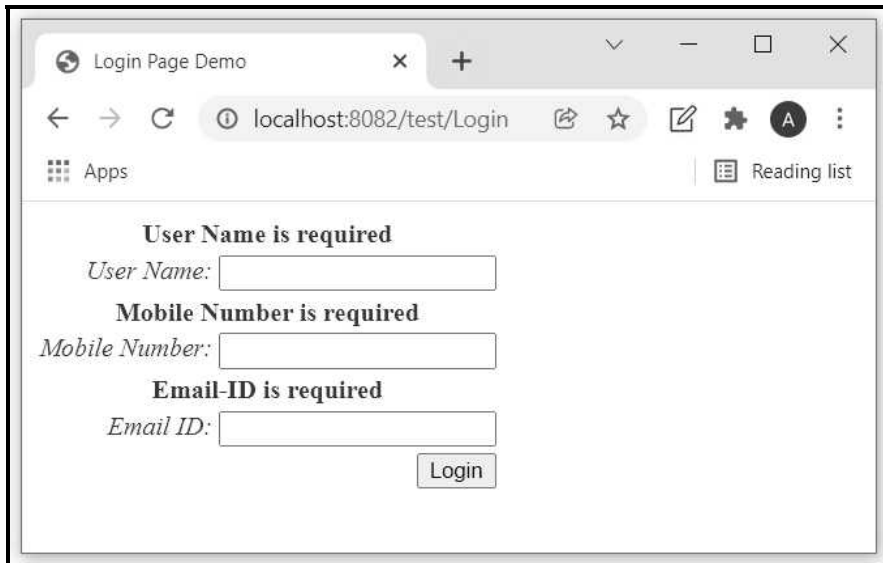
    <filter-name>struts2</filter-name>
    <url-pattern>/*</url-pattern>
  </filter-mapping>

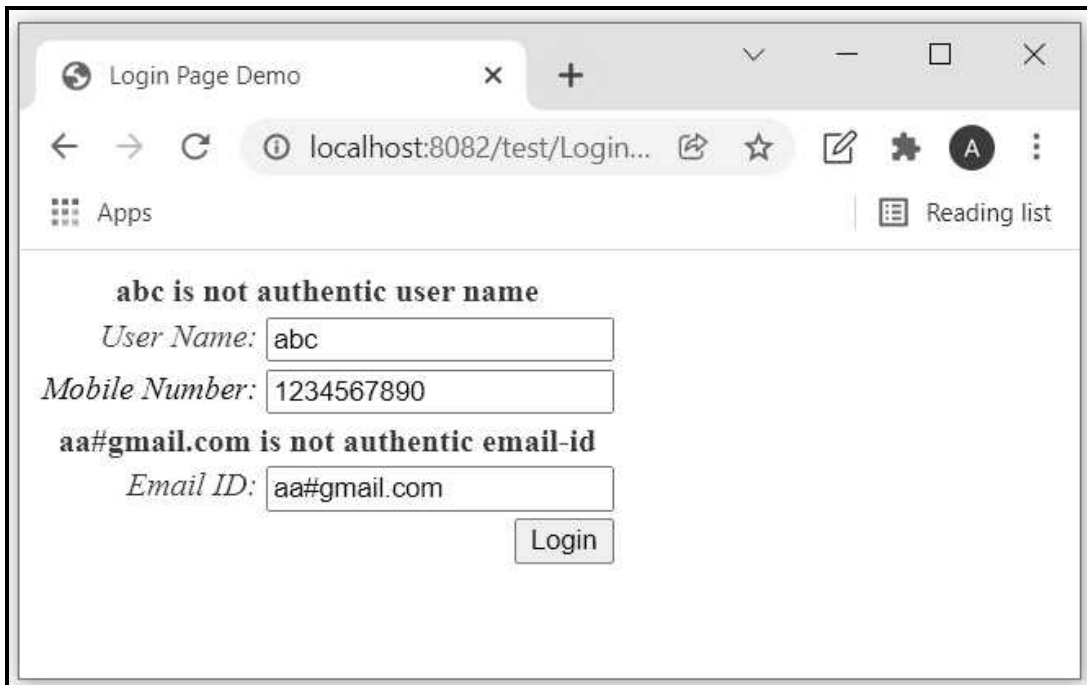
</web-app>
```

Step 6 : For executing the above application right-click on project name(here it is **test**) in the Project Explorer window, click on **Run As->Run Configuration**. Set the tomcat(under Apache) which is available on your machine.

Then right click on the project name and click **Export > WAR** File to create a War file. Then deploy this WAR in the Tomcat's webapps directory. For a **destination** click the **Browse** button and select the path `c:/your_tomcat_directory/webapps` folder by accepting the default name of the war file. Normally "your_project_name.war" file gets generated. Here we get **test.war** in the `c:/tomcat_directory/webapps`. The output can be viewed on a browser window by typing the URL for localhost. Note that as my tomcat is running on port 8082(default is 8080 port, but you can change the port number if you want), I am giving the URL with **localhost:8082**

We will first check various invalid entries. Refer following screenshots -

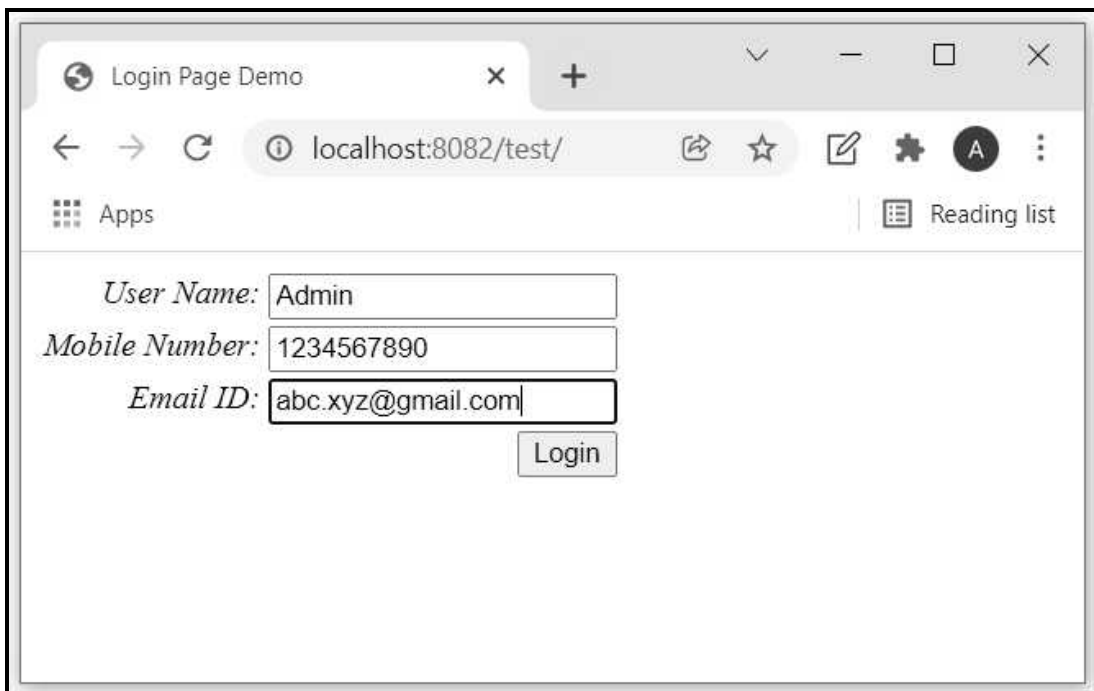




The screenshot shows a web browser window titled "Login Page Demo". The address bar displays "localhost:8082/test/Login...". The page content includes the following:

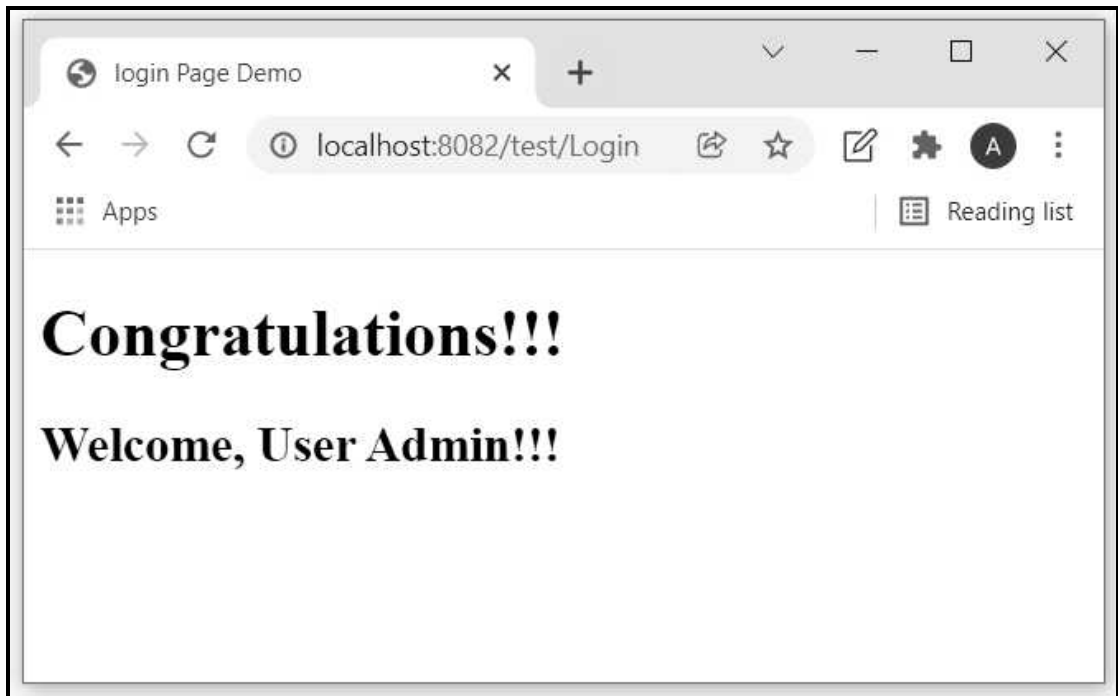
- abc is not authentic user name**
- User Name:*
- Mobile Number:*
- aa#gmail.com is not authentic email-id**
- Email ID:*
-

Now we will enter all the authentic and valid entries and get the welcome page



The screenshot shows the same web browser window, but with the following entries:

- User Name:*
- Mobile Number:*
- Email ID:*
-



Experiment 7 : Design an application using Angular JS. e.g., Design registration (first name, last name, username, password) and login page using Angular JS.

Solution :

Step 1 : Create a javascript(.js) that defines the configurations and controllers written using AngularJS. This file directs to respective homepage, registration or login page, when the link for homepage, registration page or login page is clicked.

appDemo.js

```
var app = angular.module('myApp', ['ngRoute']);
app.config(function($routeProvider) {
  $routeProvider
    .when('/', {
      templateUrl : 'home.html',
      controller : 'HomeController'
    })
    .when('/registration', {
      templateUrl : 'registration.html',
      controller : 'RegistrationController'
    })
    .when('/login', {
      templateUrl : 'login.html',
```

```
controller : 'LoginController'
})
.otherwise({redirectTo: '/'});
});
app.controller('HomeController', function($scope) {
$scope.message = 'Welcome to a home page';
});

app.controller('RegistrationController', function($scope) {
$scope.message = 'Please fill up the Registration form(New User)';
});
app.controller('LoginController', function($scope) {
$scope.message = 'Please fill up the Login form(Existing User)';
});
```

Step 2 : Create a view in HTML file. The AngularJS script is as follows -

appDemo.html

```
<!DOCTYPE html>
<html ng-app="myApp">
<head>
<meta charset="utf-8">
<meta name="viewport" content="width=device-width, initial-scale=1">
<link rel="stylesheet"
href="https://maxcdn.bootstrapcdn.com/bootstrap/3.3.7/css/bootstrap.min.css">
<script src="https://ajax.googleapis.com/ajax/libs/jquery/3.2.1/jquery.min.js"></script>
<script src="https://maxcdn.bootstrapcdn.com/bootstrap/3.3.7/js/bootstrap.min.js"></script>
<script src="https://cdnjs.cloudflare.com/ajax/libs/angular.js/1.4.7/angular.min.js"></script>
<script src="https://cdnjs.cloudflare.com/ajax/libs/angular.js/1.4.7/angular-route.min.js"></script>
</head>
<body ng-app="myApp">
<div class = "row">
<div class = "col-sm-12 well">
<center><h1>OUR WEB SITE</h1></center>
</div>
</div>
<div class="row">
<div class="col-sm-4 well" style = "background-color:pink;height:500px">
<ul>
<li><a href="#">Home</a></li>
<li><a href="#/registration">Registration</a></li>
<li><a href="#/login">Login</a></li>
</ul>
```

```
</div>
<div class="col-sm-8 well" style="background-color:aqua;height:500px">
<div ng-view></div>

</div>
</div>
<script src="appDemo.js"></script>
</body>
</html>
```

Step 3 : Now the HTML documents for home page, registration page and login page is as follows.

home.html

```
<div>
<center><h3>{{message}}</h3></center> ←Message for home page
</div>

<div>
<center>
 ←Displaying image on welcome page
</center>
</div>
```

registration.html

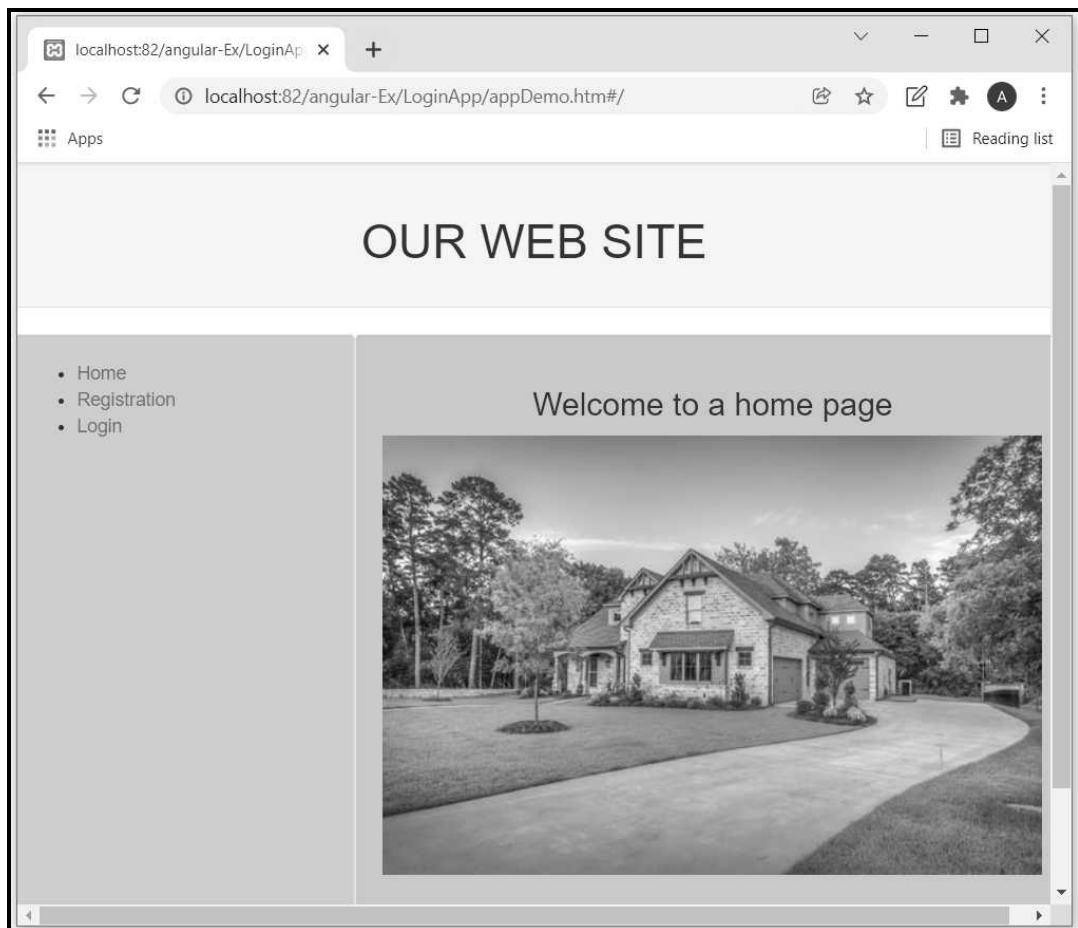
```
<html>
<head>
<title>
Registration Form
</title>
</head>
<body>
<h3>{{message}}</h3>
<form>
<p>First Name:<input type="text" name="fname"></p>
<p>Last Name:<input type="text" name="lname"></p>
<p>User Name:<input type="text" name="username"></p>
<p>Password:<input type="password" name="password"></p>
<p><input type="submit" value="submit"></p>
</form>
</body>
</html>
```

login.html

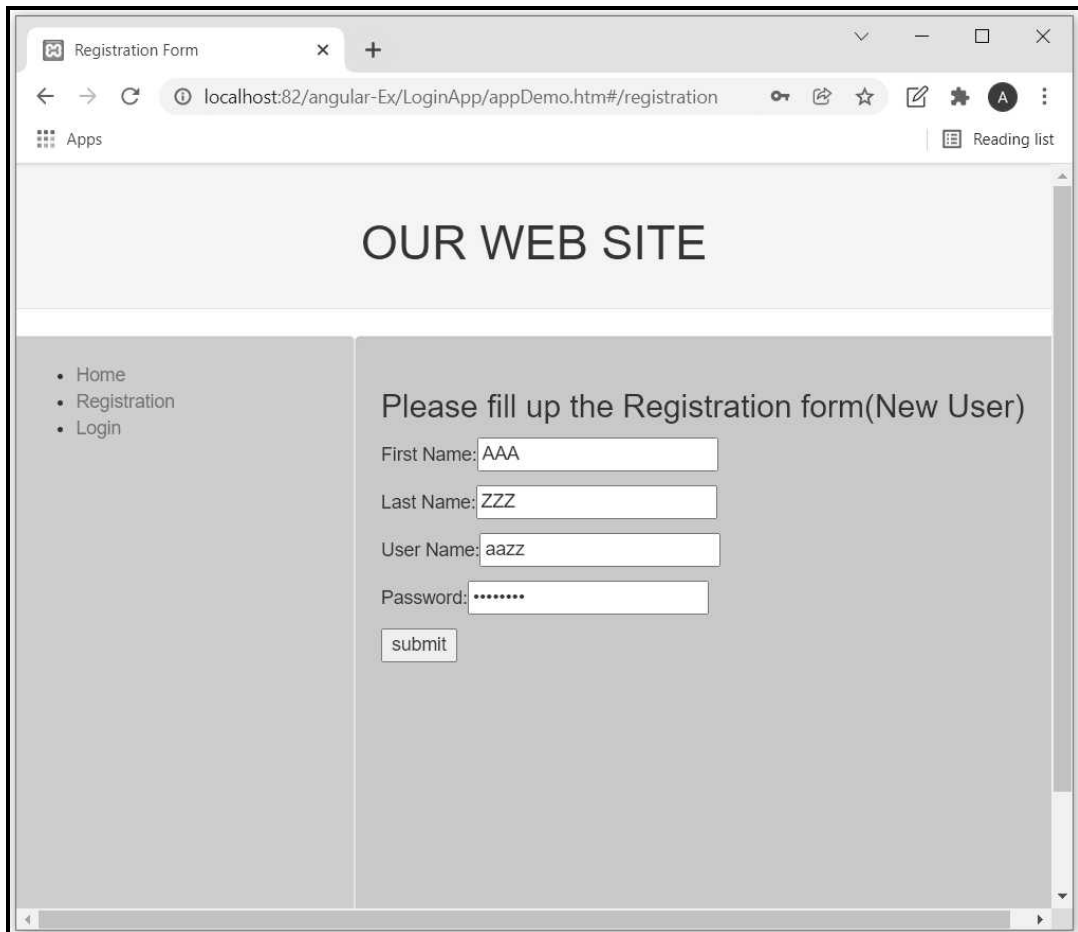
```
<html>
<head>
```

```
<title>
Login Form
</title>
</head>
<body>
<h3>{{message}}</h3>
<form>
<p>User Name:<input type="text" name="username"></p>
<p>Password:<input type="password" name="password"></p>
<p><input type="submit" value="submit"></p>
</form>
</body>
</html>
```

Step 4 : Make a separate directory and store above files (from step 1 to step 3) in it. It is preferable to store this folder in your Apache's htdocs folder. Open the web browser and give the URL for **appDemo.html** document. The output can be viewed as follows -



If we click on Registration link on the left panel we get following registration form.



The screenshot shows a web browser window with the title "Registration Form". The address bar displays "localhost:82/angular-Ex/LoginApp/appDemo.htm#/registration". The page content includes a header "OUR WEB SITE" and a left sidebar with a menu: Home, Registration, and Login. The main content area is titled "Please fill up the Registration form(New User)" and contains the following form fields:

- First Name:
- Last Name:
- User Name:
- Password:

A "submit" button is located below the password field.

If we click on Login link present in the left pane then we get following login form.

The screenshot shows a web browser window with the title 'Login Form'. The address bar displays 'localhost:82/angular-Ex/LoginApp/appDemo.htm#/login'. The page layout includes a left sidebar with a list of links: 'Home', 'Registration', and 'Login'. The main content area features a large heading 'OUR WEB SITE' and a login form titled 'Please fill up the Login form(Existing User)'. The form contains two input fields: 'User Name: aazz' and 'Password:,', followed by a 'submit' button.

Experiment 8 : Design and implement a business interface with necessary business logic for any web application using EJB. e.g., Design and implement the web application logic for deposit and withdraw amount transactions using EJB.

Solution : Refer section 6.17.4.

□□□